

به نام خدا

درس سیستم های نهفته و بی درنگ

گزارش تمرین اول

اعضای گروه:

سید نوید هاشمی - ۸۱۰۱۰۱۵۴۹

سید محمد جزایری - ۸۱۰۱۰۱۳۹۹

امیرارسلان شهبازی - ۸۱۰۱۰۱۴۵۱

سید محمد حسین مظهری - ۸۱۰۱۰۱۵۲۰

بهار ۱۴۰۵

پیاده سازی پروتکل آدرس محور (I2C).....	۴
توضیح کد	۴
سوالات	۱۳
۱. با توجه به اینکه I2C ، نسخه ی اصلی که در درس با آن آشنا شده اید، یک پروتکل باس است، در صورت اتصال بیشتر از یک سنسور با آدرس های مشابه چه اشکالی پیش می آید؟	۱۳
۲. چهار مورد از اشکالات کلی این پروتکل ساده سازی شده نسبت به آنچه از I2C در درس آموخته اید بیان کنید.	۱۳
۳. کارکرد کلاس های ByteStream, ByteVector و توابع getByte را توضیح دهید.	۱۳
پیاده سازی پروتکل USART	۱۵
توضیح کد	۱۵
سوالات	۱۷
۱. چرا بیت های شروع و پایان با بیت های داده اشتباه گرفته نمی شوند؟	۱۷
۲. با توجه به نبود کلاک (Clock) مشترک، این پروتکل چه طور داده ها را تشخیص می دهد؟	۱۸
۳. آیا امکان پیاده سازی نرم افزاری (Bit-banging) پروتکل USART وجود دارد؟	۱۹
پیاده سازی یک I2C Multiplexer	۲۰
توضیح کد	۲۰
..... Microcontroller.h	۲۰
..... I2CMux.h	۲۴
..... I2CMultiplexer.h	۳۵
..... Sensor.h	۴۲
..... final_main.cpp	۴۴
سوالات	۵۰
۱. تابع reverse در هدر Byte.h و مکان های استفاده از آن را مطالعه کنید .کارکرد آن را توضیح دهید و بگویید آیا از همچین عملیاتی در بوردهای واقعی استفاده می شود ؟ راه حل دنیای واقعی آن چیست ؟(بیان عمیق مشکل در صورت عدم ارائه راه حل دنیای واقعی نیز قابل پذیرش است)	۵۰
۲.توابع takeNthBit در Bit.h را توضیح دهید. تفاوت این دو در چیست ؟	۵۳

۳.تابع attachPinToCommunicationClock از کلاس Board توضیح دهید(از جهت کد و کاربرد) ۵۶

این پروژه حاصل کاری گروهی است. بخش اول این پروژه را سید نوید هاشمی، بخش دوم را سید محمد جزایری و بخش پایانی را امیرارسلان شهبازی و سیدمحمدحسین مظهری انجام داده‌اند.

پیاده سازی پروتکل آدرس محور (I2C)

توضیح کد

```
setup(Boards::Esp8266::Gpio& gpio) override
{
    std::cout << "esp8266 setup completed." << std::endl;
    node()->i2c.init(0x29);
    node()->i2c.start_communicating(true);
    // delay(1'000);
    node()->usart.start();
    return 0;
}
```

در ابتدا در فایل Microcontroller.h تابع setup، تابع init را برای پروتکل I2C صدا می زنیم و آدرس سنسوری که میکروکنترلر باید با آن ارتباط برقرار کند را برایش ارسال می کنیم. سپس در کلاس I2C مربوط به میکروکنترلر (در فایل Esp8266.h) تابع init را تعریف می کنیم به گونه ای که تعدادی از متغیرها را برای ادامه کار مقداردهی اولیه کند و همچنین پیامی مبنی بر راه اندازی پروتکل I2C نمایش دهد.

```
void init(Byte address) override
{
    start_communication = false;
    address_sent = false;

    pendingByte = address;

    waitingForAck = false;
    ackReceived = false;

    byteCount = 0;

    std::cout << "[ESP] Init I2C" << std::endl;
}
```

سپس در فایل Microcontroller.h تابع loop ما باید از توابعی که در کلاس I2C در فایل Esp8266.h داریم استفاده کنیم، در تصویر زیر قسمتی از loop که مربوط به استفاده از پروتکل I2C هست را مشاهده می کنیم و من در ادامه توضیح آن را می دهم.

```
// -----I2C-----
if(node()->i2c.start_communication && !node()->i2c.address_sent)
{
    node()->i2c.send_address();
}

if(node()->i2c.waitingForAck && !node()->i2c.ackReceived)
{
    node()->i2c.wait_for_recieving_ACK();
}

if(node()->i2c.ackReceived && node()->i2c.byteCount < 3)
{
    node()->i2c.recieve_3_bytes_of_data();
}
// delay(100);
return 0;
```

همانگونه که از نام متغیرها مشخص است، ما در ابتدا بررسی می‌کنیم اگر communication شروع شده بود و آدرس به سنسور ارسال نشده بود، آدرس را برای سنسور ارسال می‌کنیم و وارد استیت بعد می‌شویم؛ در استیت بعد تا زمانی که ACK را از سنسور دریافت نکرده ایم، در آن استیت منتظر دریافت ACK می‌مانیم و به محض دریافت آن، به استیت بعد می‌رویم؛ در استیت بعد وقتی که ACK دریافت شد ما باید به ترتیب سه بایت داده دریافت کنیم پس با استفاده از متغیر byteCount این بررسی را انجام می‌دهیم. اما اکنون که استفاده از I2C را در تابع loop در Microcontroller.h دیدیم بریم منطق آن را در کلاس I2C در Esp8266.h بررسی کنیم.

پس در این قسمت کدهای موجود در کلاس I2C در فایل Esp8266.h را بررسی میکنیم.

```
class I2C : public Protocols::AbstractI2C<Esp8266, Gpio>
{
public:
    explicit I2C(Esp8266* b) :
        Protocols::AbstractI2C<Esp8266, Gpio> {b}
    {}

    Byte pendingByte{}; // for sending address of the sensor on sda
    Byte currentByte{}; // it holds current recieving byte when it should recieve 3 bytes of data from the sensor

    bool start_communication = false; // this is for starting a communication that is used in loop function in Microcontroller.h
    bool address_sent = false; // this is used in loop function in Microcontroller.h

    bool waitingForAck = false; // this is used in loop function in Microcontroller.h
    bool ackReceived = false; // this is used in loop function in Microcontroller.h

    Byte receivedBytes[3]; // this array holds 3 recieving bytes of data from the sensor
    int byteCount = 0; // this is for counting recieving bytes and used in loop function in Microcontroller.h
```

در ابتدا متغیرهای کمکی را تعریف کردم و توضیحات آنها جلوی هر کدام وجود دارد.

در ادامه تعدادی تابع کمکی را در کنار چهار تابع موجود (یعنی init, write, read, run) تعریف کردم که به آنها می‌پردازیم.

```

void send_address()
{
    write(pendingByte);
    std::cout << "[ESP] Sending address: "
              << (int)pendingByte << std::endl;
    m_board->gpio().sda.write(Bit::X); // release line
    address_sent = true;
    waitingForAck = true;

    return;
}

void wait_for_recieving_ACK()
{
    if(m_board->gpio().sda.hasBitToRead())
    {
        Bit r = m_board->gpio().sda.readBit();

        std::cout << "[ESP] ACK received = " << (int)r << std::endl;

        ackReceived = (r == Bit::One);
        waitingForAck = false;

        if(!ackReceived)
        {
            std::cout << "[ESP] No ACK!" << std::endl;
            return;
        }
        // preparing to recieve data from the sensor
        byteCount = 0;
        currentByte = 0;
    }
}

```

در تابع `send_address` ما آدرس سنسور را که از قبل در متغیر `pendingByte` ذخیره کرده ایم را با استفاده از تابع `write` روی `sda` می نویسیم؛ در ادامه در I2C اصلی، هنگامیکه میکروکنترلر آدرس را می نویسد باید خط را `release` کند تا بتواند در ادامه بعنوان دریافت کننده `ack` را بگیرد پس به همین دلیل چون در توابع کمکی تابع `release` وجود نداشت ما با نوشتن یک بیت `X` این کار را انجام می دهیم؛ همچنین برای اینکه در تابع `loop` استتیت خود را تغییر دهیم، دو متغیر `address_sent` و `waitingForAck` را `true` می کنیم.

در تابع `wait_for_recieving_ACK` ما بررسی می کنیم اگر بیتی روی `sda` برای خواندن وجود داشت، آن را میخوانیم و سپس بررسی می کنیم اگر بیت خوانده شده ۱ بود، یعنی ما `ACK` را دریافت کرده ایم پس در واقع مقدار جدید `ackReceived` باعث تغییر استتیت در تابع `loop` می شود؛ در ادامه دو متغیر `byteCount` و `currentByte` را که برای دریافت داده استفاده می شود `initialize` می کنیم.

```

void recieve_3_bytes_of_data()
{
    if(m_board->gpio().sda.hasByteToRead())
    {
        Byte currentByte = read();

        receivedBytes[byteCount] = currentByte;
        std::cout << "[ESP] Received byte "
                  << byteCount << " = "
                  << (int)(UByte)currentByte << std::endl;

        byteCount++;
        currentByte = 0;

        if(byteCount == 3)
        {
            validate_3_bytes();
        }
    }

    return;
}

void validate_3_bytes()
{
    uint8_t b0 = (UByte)receivedBytes[0];
    uint8_t b1 = (UByte)receivedBytes[1];
    uint8_t chk = (UByte)receivedBytes[2];

    uint8_t expected =
        std::abs((int)b0 - (int)b1);

    bool ok = chk == expected;
    if(ok)
    {
        uint16_t value = ((uint16_t)b1 << 8) | b0;

        std::cout << "[ESP] VALUE = " << int(value)
                  << " (checksum OK)" << std::endl;
    }
    else
    {
        std::cout << "[ESP] Checksum FAILED! sensor="
                  << (int)chk << " should be="
                  << std::abs((int)b0 - (int)b1)
                  << std::endl;
    }
}

```

در تابع `recieve_3_bytes_of_data` بررسی می کنیم اگر بایتی برای خواندن روی `sda` وجود داشت، آن را با استفاده از تابع `read` می خوانیم و نمایش می دهیم، سپس هر بار با خواندن یک بایت، متغیر `byteCount` را یکی زیاد می کنیم تا وقتی به ۳ برسد، در تابع `loop` باعث تغییر استیت شود، همچنین هنگامی که به ۳ رسید، ما در واقع ۳ بایت داده را دریافت کرده ایم، باید بررسی کنیم بایت سوم قدرمطلق اختلاف دوبایت ابتدایی باشد (برای بررسی درستی بایت های دریافتی) و اگر چنین بود، داده

دوبایتی را با کنار هم قراردادن دوبایت اول می سازیم و آن را نمایش می دهیم؛ مسئول این کار هم تابع `validate_3_bytes` است.

```
void write(Byte byte) override
{
    m_board->gpio().sda.write(byte);
    return;
}

Byte read() override
{
    Byte r = m_board->gpio().sda.read();
    return r;
}

void run(Gpio& gpio) override
{
}
```

توابع `write` و `read` و `run` هم به این صورت هستند؛ در تابع `write` ما یک بایت داده که به این تابع پاس داده شده است را روی `sda` می نویسیم؛ در تابع `read` ما یک بایت داده را از روی `sda` می خوانیم و آن را خروجی می دهیم؛ در تابع `run` که با هر کلاک اجرا می شود هم کاری نمی کنیم. (البته در ابتدا من استفاده از توابع دیگر و کل فرایند این پروتکل را در تابع `run` پیاده سازی کرده بودم ولی با مشورت TA درس و همانگونه که در صورت پروژه آمده بود، آن را به تابع `loop` در `Microcontroller.h` انتقال دادم و صرفاً منطق پروتکل را در کلاس `I2C` نوشتم.)

اکنون که استفاده و منطق پروتکل `I2C` را در سمت میکروکنترلر دیدیم، باید همین موارد را در سمت سنسور ببینیم.

در ادامه در تصویر زیر فایل `Sensor.h` را مشاهده می کنیم که همانند کاری که برای میکروکنترلر انجام دادیم، برای سنسور هم در تابع `setup`، تابع `init` آن را صدا می زنیم (چون در آرگومان های آن یک بایت آدرس بود آدرس خودش را هم برایش میفرستم، البته همانگونه که در ادامه می بینیم من از این آدرس هیچ استفاده ای نمیکنم چراکه آدرس سنسور در خود فایل سنسور هاردکد شده است) و بقیه مراحل و تغییر استیت ها را در تابع `loop` انجام می دهیم.


```

class Sensor : public AbstractSketch<Sensors::V1530x>
{
public:
    explicit Sensor(Sensors::V1530x* node) :
        AbstractSketch<Sensors::V1530x> {node}
    {}

    std::int32_t
    setup(Sensors::V1530x::Gpio& gpio) override
    {
        node()->i2c.init(0x29);
        std::cout << "v1530x setup completed." << std::endl;
        return 0;
    }

    std::int32_t
    loop(Sensors::V1530x::Gpio& gpio) override
    {
        if(!node()->i2c.addressMatched)
        {
            node()->i2c.recieve_and_check_address();
        }
        if(node()->i2c.ackSent && !node()->i2c.dataSent)
        {
            node()->i2c.send_3_bytes_of_data();
        }

        return 0;
    }
};

```

در تابع loop همانگونه که از نام متغیر ها مشخص است، ما ابتدا در استیت اول باقی می مانیم تا آدرسی که روی sda دریافت میکنیم با آدرس خود سنسور match شود، سپس به استیت بعدی می رویم و تا زمانی که ۳ بایت داده را برای میکروکنترلر نفرستیم در همان استیت باقی می مانیم؛ در ادامه این توابع و منطق های آن ها را در فایل VL530X.h بررسی می کنیم.

```

class I2C : public Protocols::AbstractI2C<V1530x, Gpio>
{
public:
    explicit I2C(V1530x* b) :
        Protocols::AbstractI2C<V1530x, Gpio> {b}
    {}

    bool addressMatched = false; //this is used in Sensor.h for changing state
    bool ackSent = false; //this is used in Sensor.h for changing state

    //for sending bytes to the microcontroller
    Byte tx[3]; // this array is for saving 3 bytes of data that we should send to the microcontroller
    int txIndex = 0; //this is for counting sending bytes
    bool dataSent = false; //this is used in Sensor.h for changing state

```

در ابتدا متغیر های کمکی را تعریف کردم و توضیحات آنها جلوی هر کدام وجود دارد.
در ادامه تعدادی تابع کمکی را در کنار چهار تابع موجود (یعنی init, write, read, run) تعریف کردم که به آنها می پردازیم.

```

void recieve_and_check_address()
{
    if(m_board->gpio().sda.hasByteToRead())
    {
        Byte rec_byte = read();
        std::cout << "[Sensor] Received byte = "
                    << int(rec_byte) << std::endl;

        if(rec_byte == V1530x::address)
        {
            addressMatched = true;
            std::cout << "[Sensor] Address matched!" << std::endl;

            send_ACK_to_Esp();
            generate_2_bytes_and_checksum();
        }
    }

    return;
}

void send_ACK_to_Esp()
{
    Bit ack = Bit::One;
    m_board->gpio().sda.write(ack);
    std::cout << "[Sensor] ACK sent = " << (int)ack <<std::endl;
    ackSent = true;

    return;
}

```

در تابع `receive_and_check_address` بررسی میکنیم اگر بایتی روی `sda` برای خواندن وجود داشت، با استفاده از تابع `read` آن را می خوانیم و سپس بررسی میکنیم آیا این بایت خوانده شده همان آدرس سنسور است یا نه، اگر این دو با هم برابر بودند یعنی باید در ادامه `ack` و داده بفرستیم که برای این کار دو تابع `send_ACK_to_Esp` و `generate_2_bytes_of_data` را صدا می زنیم.

تابع `send_ACK_to_Esp` همانطور که از اسمش مشخص است، ۱ را بعنوان `ack` روی `sda` می نویسد و برای اینکه استیتمان در تابع `loop` در `Sensor.h` تغییر کند، متغیر `ackSent` را `true` میکند.

در ادامه همانطور که در تصویر زیر مشاهده می شود، در تابع `generate_2_bytes_and_checksum` در ابتدا به کمک تابع کمکی `make_random_distance` یک عدد رندوم بین ۰ تا ۴۰۰۰ تولید می کنیم، سپس آن را به دو قسمت تقسیم می کنیم تا بعنوان بایت های اول و دوم ارسال کنیم، همچنین قدرمطلق اختلاف آن ها را نیز بعنوان بایت سوم در ذخیره می کنیم.

همچنین تابع `send_3_bytes_of_data` نیز هر بار با توجه به مقدار متغیر `txIndex` یک بایت از آن سه بایتی که تولید کردیم را با استفاده از تابع `write` روی `sda` می نویسد و این اندیس را یکی زیاد می کند تا هنگامیکه ما ۳ بایت داده را فرستادیم، متغیر `dataSent` مقدارش `true` شود و ما در تابع `loop` در فایل `Sensor.h` از استیست ارسال داده خارج شویم.

```

void send_3_bytes_of_data()
{
    Byte curByte = tx[txIndex];
    write(curByte);

    std::cout << "[Sensor] Sending Byte " << txIndex
        << ": " << (int)(UByte)curByte << std::endl;

    txIndex++;

    if(txIndex == 3)
    {
        std::cout << "[Sensor] Transmission finished\n";
        dataSent = true;
        // addressMatched = false;
    }

    return;
}

std::uint16_t make_random_distance()
{
    static std::mt19937 rng{std::random_device{}()};
    static std::uniform_int_distribution<std::uint16_t> dist(0, 4000);
    return dist(rng);
}

void generate_2_bytes_and_checksum()
{
    std::uint16_t value = make_random_distance();
    ByteVector<std::uint16_t> vec(value);
    tx[0] = getByte<0>(vec); // LSB
    tx[1] = getByte<1>(vec); // MSB

    // checksum = |tx[0] - tx[1]|
    tx[2] = std::abs((int)(UByte)tx[0] - (int)(UByte)tx[1]);

    txIndex = 0;
    std::cout << "[Sensor] Preparing for sending data to Esp." << std::endl;
}

```

در ادامه نیز چهار تابع run , read , write , init را مشاهده می کنیم.

در تابع init متغیر هایی که در طول اجرای پروتکل برای تغییر استیت ها و موارد دیگر استفاده می کنیم را مقداردهی اولیه می کنیم.

در تابع write بایت داده ای که بعنوان ورودی به تابع وارد می شود را روی sda می نویسیم.

در تابع read یک بایت داده از روی sda می خوانیم و آن را خروجی می دهیم.

در تابع run هم مانند تابع متناظرش در فایل Esp8266.h کاری انجام نمی دهیم.

```

void init(Byte address) override
{
    srand(time(nullptr));

    addressMatched = false;
    ackSent = false;
    dataSent = false;
}

void write(Byte byte) override
{
    m_board->gpio().sda.write(byte);
    return;
}

Byte read() override
{
    Byte r = m_board->gpio().sda.read();
    return r;
}

void run(Gpio& gpio) override
{
}

```

در ادامه نمونه ای از خروجی اجرای این پروتکل آورده ام:

```

Esp8266::Processor created.
one instance of Esp8266 created.
VL530x::processor created.
one instance of VL530x created.
linkRed: header Esp8266::vdd1 attached to link successfully.
Esp8266::vdd1: linkRed attached successfully
linkBlack: header Esp8266::gnd1 attached to link successfully.
Esp8266::gnd1: linkBlack attached successfully
linkGreen: header Esp8266::scl attached to link successfully.
Esp8266::scl: linkGreen attached successfully
linkYellow: header Esp8266::sda attached to link successfully.
Esp8266::sda: linkYellow attached successfully
linkRed: header VL530x::vdd attached to link successfully.
VL530x::vdd: linkRed attached successfully
linkBlack: header VL530x::gnd attached to link successfully.
VL530x::gnd: linkBlack attached successfully
linkGreen: header VL530x::scl attached to link successfully.
VL530x::scl: linkGreen attached successfully
linkYellow: header VL530x::sda attached to link successfully.
VL530x::sda: linkYellow attached successfully
esp8266 setup completed.
[ESP] Init I2C
[ESP] Sending address: 41
vl530x setup completed.
[Sensor] Received byte = 41
[Sensor] Address matched!
[Sensor] ACK sent = 1
[Sensor] Preparing for sending data to Esp.
[Sensor] Sending Byte 0: 22
[ESP] ACK received = 1
[Sensor] Sending Byte 1: 11
[ESP] Received byte 0 = 22
[Sensor] Sending Byte 2: 11
[Sensor] Transmission finished
[ESP] Received byte 1 = 11
[ESP] Received byte 2 = 11
[ESP] VALUE = 2838 (checksum OK)

```

سوالات

۱. با توجه به اینکه I2C ، نسخه‌ی اصلی که در درس با آن آشنا شده‌اید، یک پروتکل باس است، در صورت اتصال بیشتر از یک سنسور با آدرس‌های مشابه چه اشکالی پیش می‌آید؟

در پروتکل I2C اصلی، هر وسیله ای که روی باس قرار می‌گیرد باید یک آدرس یکتا داشته باشد تا وقتی میکروکنترلر روی خط SDA آدرس را ارسال می‌کند، فقط همان وسیله پاسخ دهد. اگر دو یا چند سنسور آدرس یکسان داشته باشند، هر دو همزمان به فریم آدرس واکنش نشان می‌دهند و این باعث تداخل می‌شود. این تداخل معمولاً به این شکل خودش را نشان می‌دهد که هر دو سنسور تلاش می‌کنند در یک لحظه روی خط SDA داده بفرستند یا ACK بدهند و در نتیجه سیگنال روی خط به هم می‌ریزد و میکروکنترلر نمی‌تواند تشخیص دهد که پاسخ درست از کدام ماژول آمده است. در نهایت ارتباط پایدار شکل نمی‌گیرد و معمولاً خواندن داده‌ها یا انجام عملیات بعدی به طور کامل مختل می‌شود.

۲. چهار مورد از اشکالات کلی این پروتکل ساده‌سازی شده نسبت به آنچه از I2C در درس آموخته‌اید بیان کنید.

در این نسخه ساده شده، بخش زیادی از پیچیدگی‌های I2C واقعی حذف شده و همین موضوع باعث می‌شود بعضی قابلیت‌های مهم آن از بین برود. در ادامه چهار مورد از اشکالات آن را بطور خلاصه می‌گوییم:

(۱) در این نسخه ساده شده امکان وجود چند وسیله روی باس در نظر گرفته نشده است. در I2C واقعی مکانیزم Arbitration و Addressing دقیق وجود دارد تا چندین سنسور و ماژول بتوانند همزمان روی یک زوج سیم کار کنند، اما در این پیاده‌سازی کوچک‌ترین تداخل باعث از کار افتادن ارتباط می‌شود.

(۲) مورد بعدی حذف شدن مکانیزم‌های کنترل خطا است. در I2C اصلی علاوه بر ACK/NACK، شرایطی مثل repeated start، clock stretching و ... وجود دارد که کمک می‌کنند خطاهای لحظه‌ای یا تاخیرهای زمانی مدیریت شوند. در این نسخه همچنین چون ما کنترلی روی SCL نداریم، هیچ‌کدام از این توانایی‌ها وجود ندارد و ارتباط بسیار شکننده‌تر می‌شود.

(۳) هماهنگی زمانی در I2C واقعی با دقت بالایی مدیریت می‌شود. سنسور می‌تواند در صورت نیاز clock را نگه دارد (clock stretching) یا باس را آزاد کند اما در این نسخه، این هماهنگی‌ها حذف شده و تمام مسئولیت زمان بندی روی دوش میکروکنترلر است، بنابراین اگر کوچک‌ترین اختلاف زمانی ایجاد شود، داده‌ها درست منتقل نمی‌شوند.

(۴) پروتکل اصلی از نظر ساختار پیام و نحوه مشخص کردن عملیات خواندن و نوشتن بسیار کامل‌تر است. در این نسخه فقط یک الگوی حداقلی استفاده شده و امکاناتی مثل ارسال پشت سر هم داده، آدرس دهی رجیسترهای داخلی، و مدیریت فریم‌های طولانی وجود ندارد که این موضوع ارتباط را به شدت محدود می‌کند.

۳. کارکرد کلاس‌های `ByteStream`، `ByteVector` و `getByte` را توضیح دهید.

کارکرد کلاس `ByteStream`:

این کلاس در واقع برای زمانی طراحی شده که قرار باشد چند بایت جدا از هم به صورت ترتیبی دریافت شود و در نهایت از ترکیب آن ها یک مقدار بزرگ تر ساخته شود. به بیان ساده تر، این کلاس مثل یک جریان ورودی بایت عمل می کند.

وقتی یک بایت جدید با عملگر >> به آن داده می شود، این بایت به داخل متغیر داخلی شیفت داده می شود و مقدار قبلی هم به تعداد ۸ بیت (اندازه ی Byte) به چپ منتقل می گردد تا جا برای بایت جدید باز شود. به این ترتیب، اگر برای مثال سه بایت پشت سر هم دریافت شده باشند، این سه بایت به همان ترتیب درون متغیر نهایی قرار می گیرند.

کلاس همچنین تعداد بایت هایی را که تا آن لحظه دریافت کرده نگه می دارد و از این طریق می تواند تشخیص بدهد که آیا تعداد بایت های لازم برای تشکیل مقدار اصلی کامل شده یا نه. تابع isReady همین را بررسی می کند. وقتی تعداد بایت ها کامل شد، تابع های value یا take مقدار نهایی را برمی گردانند. تفاوت value و take در این است که take بعد از خواندن مقدار، جریان را پاک می کند تا دوباره قابل استفاده شود.

به طور خلاصه این کلاس مسئول ساختن مقدار های چند بایتی از روی جریان بایت های ورودی است و جلوی بروز overflow را هم می گیرد.

کارکرد کلاس ByteVector:

این کلاس به نوعی نقش برعکس ByteStream را دارد. یعنی به جای این که بخواهد چند بایت را کنار هم بگذارد، وظیفه اش این است که یک مقدار چند بایتی را به شکل مجموعه ای از بایت های جدا از هم در دسترس قرار بدهد.

مقداری که به constructor کلاس داده می شود، داخل object ذخیره شده و سپس با reinterpret_cast همان مقدار به شکل یک آرایه ای از Byte دیده می شود. به این ترتیب، با استفاده از عملگر [] می توان هر بایت از مقدار اصلی را به ترتیب در حافظه خواند.

بطور کلی می شود گفت ByteVector ابزاری است برای دیدن یک مقدار چند بایتی در قالب آرایه ای از بایت ها. این قابلیت وقتی به کار می آید که بخواهیم یک مقدار عددی را بایت بایت روی خط ارسال کنیم.

کارکرد توابع getByte:

این توابع در واقع مکمل ByteVector هستند. کاری که آنها انجام می دهند این است که از روی یک مقدار عددی، یا از روی یک ByteVector، یکی از بایت های آن مقدار را بر اساس شماره ای که به تابع داده شده استخراج کنند.

این توابع برای انواع lvalue و rvalue مقدار را پوشش می دهند و با یک reinterpret_cast ساده، مقدار اصلی را به صورت آرایه ای از بایت می بینند و سپس بایت مورد نظر را بر می گردانند. همچنین در همه نسخه ها با static_assert بررسی می شود که شماره بایت از اندازه واقعی مقدار بیشتر نباشد.

به این ترتیب getByte یک روش ساده برای گرفتن بایت های یک مقدار چند بایتی است، بدون این که بخواهیم خودمان محاسبه شیفت و ماسک انجام بدهیم.

پیاده سازی پروتکل USART

توضیح کد

```
#ifndef HARDDISK_H
class HardDisk : public AbstractSketch<Sensors::Usb>
{
    std::int32_t
    loop(Sensors::Usb::Gpio& gpio) override
    {
        //2
        if(!node()->usart.has_sensed_zero()) {
            node()->usart.set_sensed_zero();
        }
        //3
        if(!node()->usart.has_received_byte() && node()->usart.has_sensed_zero()) {
            address = node()->usart.read();
        }
        //4
        if(!node()->usart.has_sensed_one() && node()->usart.has_received_byte() &&
            node()->usart.has_sensed_zero()) {
            node()->usart.set_sensed_one();
        }
        //5
        if(node()->usart.has_received_byte() && node()->usart.has_sensed_one()) {
            std::cout << "Disk::Received address is:" << (int)address << " and sent data is: " << (int)m_storage[address] << std::endl;
            node()->usart.write(m_storage[address]);
            node()->usart.reset_vars();
        }
        return 0;
    }
}
```

در تصویر فوق تابع loop کلاس هارد دیسک را مشاهده می کنیم. در if اول بررسی می کنیم که آیا مقدار صفر روی لینک tx نوشته شده یا نه تا شروع ارسال داده را تشخیص دهیم. در if دوم که به شرط گرفتن صفر برقرار می شود منتظر نوشته شدن یک بایت روی لینک می مانیم و سپس آن را برمیداریم، این عدد آدرس حافظه موردنظر کنترلر است.

در if سوم و پس از دریافت یک بایت داده منتظر می مانیم که لینک دوباره یک شود تا از صحت داده های دریافتی مطمئن شویم و در if آخر داده موردنظر کنترلر را بر روی لینک rx می نویسیم و با تابع reset_vars تمام متغیرهای پروتکل را به حالت اولیه برمیگردانیم تا اتمام پروتکل را اعلام کنیم.

```

#ifndef MICROCONTROLLER_H
class MicroController : public AbstractSketch<Boards::Esp8266>
{
    setup(Boards::Esp8266::Gpio& gpio) override
    {
        node()->usart.write(0x22);
        // delay(1'000);
        return 0;
    }

    std::int32_t
    loop(Boards::Esp8266::Gpio& gpio) override
    {
        //6
        if(!node()->usart.has_sensed_zero()) {
            node()->usart.set_sensed_zero();
        }
        //7
        if((!node()->usart.has_received_byte()) && node()->usart.has_sensed_zero()) {
            data = node()->usart.read();
            std::cout << "Esp::Received data is:" << (int)data << std::endl;
        }
        //8
        if(!node()->usart.has_sensed_one() && node()->usart.has_received_byte() &&
            node()->usart.has_sensed_zero()) {
            node()->usart.set_sensed_one();
        }
        //9
        if(node()->usart.has_received_byte() && node()->usart.has_sensed_one()) {
            std::cout << "Esp::done!" << std::endl;
            node()->usart.reset();
        }
    }
};

```

در این تصویر اجرای پروتکل در سمت کنترلر را می بینیم. ابتدا در تابع setup داده با آدرس ۳۴ را درخواست می کنیم سپس در اولین if تابع loop منتظر صفر شدن لینک rx می مانیم تا آغاز ارسال داده را اعلام کند سپس اگر صفر را دیدیم از روی لینک یک بایت کی خوانیم که همان داده موجود در خانه حافظه شماره ۳۴ است و در مرحله بعدی منتظر بیت یک می مانیم تا از صحت اجرای پروتکل و درستی داده مطمئن شویم. در آخرین گام متغیرهای پروتکل را به حالت اولیه برمی گردانیم.

```

#ifndef ESP8266_H
namespace Boards
{
class Esp8266
{
    class USART : public Protocols::AbstractUsart<Esp8266, Gpio>
    {
    public:
        void
        write(Byte byte) override
        {
            Esp->gpio().tx.write(Bit::Zero);
            Esp->gpio().tx.write(byte);
            Esp->gpio().tx.write(Bit::One);
            return;
        }

        Byte
        read() override
        {
            if(Esp->gpio().rx.hasByteToRead()){
                frame = Esp->gpio().rx.read();
                received_byte = true;
                return frame;
            }
            return 0;
        }

        void set_sensed_zero() {
            if(Esp->gpio().rx.hasBitToRead()) {
                if(Esp->gpio().rx.readBit() == Bit::Zero) {
                    sensed_zero = true;
                }
            }
        }

        void set_sensed_one() {
            if(Esp->gpio().rx.hasBitToRead()) {
                if(Esp->gpio().rx.readBit() == Bit::One) {
                    sensed_one = true;
                }
            }
        }

        void reset() {
            sensed_zero = false;
            received_byte = false;
            sensed_one = false;
        }
    };
};

```


در تصویر بالا توابعی که سمت ESP8266 پیاده سازی شده‌اند را مشاهده می‌کنیم در تابع write روی tx می‌نویسیم و برای خواندن بایت و دو بیت قبل و بعد از یک بایت از rx می‌خوانیم در حالی که همانطور که در تصویر بعدی می‌بینیم برای USB برعکس است.

```
#ifndef USB_H
namespace Sensors {
class Usb : public Board<BaudRates::NotSpecified,
class USART : public Protocols::AbstractUsart<Usb, Gpio>
reset_vars()

void
write(Byte byte) override
{
    m_usb->gpio().RX.write(Bit::Zero);
    m_usb->gpio().RX.write(byte);
    m_usb->gpio().RX.write(Bit::One);
    return;
}
Byte
read() override
{
    if(m_usb->gpio().TX.hasByteToRead()){
        frame = m_usb->gpio().TX.read();
        received_byte = true;
        return frame;
    }
    return 0;
}
void set_sensed_zero() {
    if(m_usb->gpio().TX.hasBitToRead()){
        if(m_usb->gpio().TX.readBit() == Bit::Zero) {
            sensed_zero = true;
        }
    }
}
void set_sensed_one() {
    if(m_usb->gpio().TX.hasBitToRead()) {
        if(m_usb->gpio().TX.readBit() == Bit::One) {
            sensed_one = true;
        }
    }
}
}
```

در تصویر پایین تابع main را می‌بینیم.

```
USBnode.gpio().vdd.attachLink(disk_and_usb_vdd);
USBnode.gpio().gnd.attachLink(disk_and_usb_gnd);
USBnode.gpio().TX.attachLink(TX);
USBnode.gpio().RX.attachLink(RX);
USBnode.gpio().RX.setCanRead(false); // USB CAN'T READ FROM THIS PIN!

esp8266.gpio().tx.attachLink(TX);
esp8266.gpio().tx.setCanRead(false); // MICROCONTROLLER CAN'T READ FROM THIS PIN!
esp8266.gpio().rx.attachLink(RX);

micro.start();
my_hard_disk.start();
disSen.start();
```

دو خط که با کامنت تعیین شده‌اند در عملکرد درست پروتکل بسیار مهم هستند! خط اول می‌گوید که USB حق ندارد از لینک rx بخواند و خط دوم می‌گوید که ESP8266 حق ندارد از tx بخواند.

سوالات

۱. چرا بیت‌های شروع و پایان با بیت‌های داده اشتباه گرفته نمی‌شوند؟

دلیل اصلی این تمایز، وجود قوانین زمان‌بندی (Timing) و تغییر حالت منطقی (Edge Detection) است.

بیت شروع (Start Bit) در حالت عادی (زمانی که هیچ داده‌ای در حال ارسال نیست)، خط TX روی سطح منطقی بالا قرار دارد. بیت شروع همیشه با تغییر ناگهانی از حالت ۱ به ۰ مشخص می‌شود. گیرنده با مشاهده این لبه نزولی متوجه می‌شود که یک فریم جدید در حال شروع است. هیچ داده‌ای با سطح ۰ شروع نمی‌شود مگر اینکه خودِ بیت شروع باشد، زیرا گیرنده انتظار دارد اگر لبه نزولی دید، شروع یک بایت جدید است.

بیت‌های داده (Data Bits) این بیت‌ها بلافاصله پس از بیت شروع و با فواصل زمانی مشخص ارسال می‌شوند. گیرنده با شمارش تعداد پالس‌های کلاک داخلی که با نرخ Baud Rate همگام‌سازی شده است موقعیت هر بیت داده را تشخیص می‌دهد.

بیت پایان (Stop Bit) بیت پایان همیشه مقدار ۱ دارد. وقتی گیرنده یک بایت داده را دریافت کرد، به طور خودکار می‌داند که بیت بعدی باید بیت پایان باشد. اگر این بیت ۱ باشد، فریم سالم است. اگر گیرنده در موقعیت بیت پایان، مقدار ۰ را مشاهده کند، ارسال به درستی رخ نداده است.

بنابراین، ساختار فریم (Start-Data-Stop) و زمان‌بندی دقیق باعث می‌شود که هر بیت در جایگاه مشخص خود قرار گیرد و تداخلی با هم نداشته باشند.

۲. با توجه به نبود کلاک (Clock) مشترک، این پروتکل چه‌طور داده‌ها را تشخیص می‌دهد؟

پروتکل USART یک پروتکل ناهمزمان (Asynchronous) است، به این معنی که کلاک فیزیکی مشترکی بین فرستنده و گیرنده وجود ندارد. اما داده‌ها به درستی تشخیص داده می‌شوند به دلیل دو عامل کلیدی:

۱. همگام‌سازی نرخ داده (Baud Rate Agreement) قبل از شروع ارتباط، هر دو طرف (میکروکنترلر و ماژول/هارد دیسک) باید روی یک نرخ انتقال داده (Baud Rate) توافق کنند. این نرخ تعیین می‌کند که هر بیت چقدر طول می‌کشد (مثلاً در ۹۶۰۰ باود، هر بیت حدود ۱۰۰ میکروثانیه طول می‌کشد). اگر نرخ کلاک داخلی هر دو دستگاه بسیار به هم نزدیک باشد، گیرنده می‌تواند بیت‌ها را در زمان مناسب نمونه‌برداری (Sample) کند.

۲. نمونه‌برداری در وسط بیت (Mid-bit Sampling) گیرنده منتظر لبه نزولی بیت شروع می‌ماند. پس از تشخیص لبه نزولی، گیرنده یک تاخیر کوتاه (معمولاً نصف مدت زمان یک بیت) ایجاد می‌کند و سپس شروع به شمارش پالس‌های کلاک داخلی خود می‌کند.

- گیرنده بیت‌های داده را در نصفه زمانی هر بیت نمونه‌برداری می‌کند، نه در لبه‌های آن.
- دلیل این کار این است که اگر نویزی در لبه بیت رخ دهد یا نویسان کمی در فرکانس کلاک بین فرستنده و گیرنده وجود داشته باشد، نمونه‌برداری در وسط بیت (که پایدارترین نقطه سیگنال است) کمترین احتمال خطا را دارد.

- با این روش، حتی اگر کلاک گیرنده کمی سریع تر یا کندتر از فرستنده باشد، تا چند بیت اول خطای چشمگیری ایجاد نمی‌شود، زیرا نمونه‌برداری در وسط بیت انجام می‌گیرد.

در نهایت، بیت شروع نقش "کلاک مرجع موقت" را بازی می‌کند و گیرنده ساعت داخلی خود را با لبه شروع بیت شروع، همگام‌سازی می‌کند تا برای دریافت بیت‌های بعدی آماده شود.

۳. آیا امکان پیاده‌سازی نرم‌افزاری (Bit-banging) پروتکل USART وجود دارد؟

بله، کاملاً امکان‌پذیر است.

پیاده‌سازی USART به صورت نرم‌افزاری که به آن Bit-banging گفته می‌شود، یک روش رایج در میکروکنترلرهایی است که ماژول سخت‌افزاری USART را ندارند، یا وقتی که پین‌های سخت‌افزاری در دسترس نیستند.

دلایل و نحوه عملکرد:

۱. کنترل مستقیم پین‌ها: در این روش، برنامه‌نویس با استفاده از توابعی مانند `digitalWrite()` و `digitalRead()` در آردوینو (یا معادل آن در C خالص)، پین‌های دیجیتال میکروکنترلر را به صورت دستی کنترل می‌کند.

- برای ارسال یک بیت، پین TX به سطح مناسب (۰ یا ۱) برده می‌شود و با یک تاخیر زمانی دقیق (مثلاً `delayMicroseconds(104)` برای ۹۶۰۰ باود) نگه داشته می‌شود.

- برای دریافت، برنامه به صورت مداوم (Polling) پین RX را چک می‌کند تا لبه نزولی (شروع بیت) را تشخیص دهد.

۲. دقت زمان‌بندی: مهم‌ترین چالش در پیاده‌سازی نرم‌افزاری، ایجاد تاخیرهای بسیار دقیق است. اگر تاخیرها دقیق نباشند، گیرنده بیت‌ها را در زمان اشتباه نمونه‌برداری کرده و داده‌ها مخدوش می‌شوند. در میکروکنترلرهایی مانند Arduino Uno (با پردازنده ATmega328P)، این کار با استفاده از توابع تاخیر نرم‌افزاری یا تایمرها قابل انجام است.

۳. مقایسه با I2C: همان‌طور که اشاره شد، I2C نیز می‌تواند به صورت نرم‌افزاری (Soft-I2C) پیاده‌سازی شود. USART نیز دقیقاً مشابه آن است. تفاوت این است که I2C نیاز به مدیریت کلاک (SCL) و داده (SDA) همزمان دارد، در حالی که USART فقط به مدیریت زمان‌بندی روی یک خط (TX) و تشخیص زمان روی خط دیگر (RX) نیاز دارد.

۴. معایب و مزایا:

- مزیت: انعطاف‌پذیری بالا؛ می‌توانیم هر پین دیجیتالی را به عنوان TX یا RX انتخاب کنیم.
- عیب: استفاده از CPU. در پیاده‌سازی نرم‌افزاری، پردازنده باید در حین ارسال/دریافت داده، کارهای دیگری انجام ندهد (یا باید از وقفه‌ها استفاده کرد)، زیرا هر خط کد باید دقیقاً در زمان مشخص اجرا شود. این موضوع

باعث می‌شود مصرف منابع پردازشی بیشتر شود و پهنای باند مؤثر کاهش یابد. همچنین، اگر پردازنده درگیر عملیات سنگین دیگری باشد، ممکن است تاخیرهای ناخواسته ایجاد شود که منجر به خطای ارتباطی می‌گردد.

نتیجه‌گیری: پیاده‌سازی نرم‌افزاری USART امکان‌پذیر است و در پروژه‌های آموزشی یا وقتی که پهنای سخت‌افزاری دسترس نیستند، بسیار مفید است. اما برای ارتباطات پرسرعت یا سیستم‌های حساس به زمان، استفاده از ماژول سخت‌افزاری USART ترجیح داده می‌شود.

پیاده‌سازی یک I2C Multiplexer

توضیح کد

ابتدا یک توضیح کلی در ارتباط با اینکه ایده از کجا خلق شد می‌دهیم سپس به توضیح جزئیات می‌پردازیم. در بخش‌های قبل ما صرفاً یک اتصال یک به یک داشتیم یعنی میکروکنترلر دستور می‌داد و سنسور یا هارددیسک پاسخ می‌دادند. اما در این بخش قصد داریم یک کار جنرال انجام دهیم. به این معنی که ترکیبی از ماژول‌ها را استفاده کنیم. بحث مالتی پلکسر یک طرف قضیه است اما بخش مهم‌تر این است که کد ما توانایی این را داشته باشد که بسته به انتخاب کاربر در هر یک از ۳ فاز قرار بگیرد یعنی مثلاً اگر کاربر خواست بخش اول را اجرا کند با تغییرات حداقلی نسبت به کد واحد بتواند مدی که در آن است را انتخاب کند. مد را حالتی که برنامه ما روی آن قرار است کار کند تعریف می‌کنیم. برای آنکه کد ما بتواند هر سه بخش پروژه را روی یک کد واحد با حداقل تغییر اجرا کند ۳ مد متفاوت برای آن تعریف می‌کنیم. مد اول برای پیاده‌سازی بخش اول (i2c)، مد دوم برای پیاده‌سازی بخش دوم (usart) و مد سوم برای بخش سوم مالتی پلکسر طراحی شده است. و در کد `final_main.cpp` متغیر `mode` تعریف شده است تا بسته به اینکه از کدام بخش می‌خواهیم اجرا بگیریم آن را تغییر دهیم.

پس ایده کلی ما از اینجا به بعد تعریف یک مد برای برخی ماژول‌هاست. در ادامه با جزئیات کار آشنا می‌شویم.

Microcontroller.h

اولین ماژول شمای بخش سوم میکروکنترلر است. همانطور که بالاتر اشاره شد کد کلی این ماژول هم در ستاپ هم لوپ از سه ایف تشکیل می‌شود که هر ایف برای یک مد است.

```
// I2C mode
if(mode == 1){ ...

// Usart mode
if(mode == 2){ ...

// I2C mux mode
if(mode == 3){ ...
```

با توجه به اینکه ارتباطات میکروکنترلر در بخش سوم بسیار شبیه بخش دوم است شباهت های بسیاری در مد ۳ و مد ۲ خواهیم داشت.

در متد ستاپ صرفا متد ستاپ پروتکل USART صدا زده می شود.

در متد لوپ نیز تقریبا مد ۲ و ۳ شبیه هم هستند با این تفاوت که در بخش سوم ما باید یک شماره کانال داشته باشیم و با عوض کردن شماره کانال و اعلام آن به ماژول I2CMux داده را از سنسور مدنظر بخوانیم.

```
if(mode == 3)
{
    static int selected_channel = 0;
    static bool channel_command_sent = false;
    static int received_byte_index = 0;
    static Byte received_sensor_packet[3]{};

    if(!channel_command_sent && node()->usart.is_ready_to_send())...

    if(channel_command_sent) ...
}
```

کد ایف مد ۳ در متد لوپ

در ابتدا یکسری متغیر در این بخش تعریف می کنیم. متغیر اول برای مشخص کردن اینکه کدام کانال را می خواهیم انتخاب کنیم.

متغیر command sent برای این است که مشخص کنیم آیا انتخاب کانال انجام شده است یا خیر. (عملا بخش اضافی نسبت به پیاده سازی مد ۲)

متغیر سوم و چهارم برای این هستند که ۳ بایتی که از سنسور به ماکس و سپس از ماکس به میکروکنترلر می رسد را ذخیره کنند و سپس از آن برای محاسبه چکسام استفاده می کنیم.

دو شرط ایف داریم که اولی اینگونه است که چک می کند میکروکنترلر آماده ارسال است اما هنوز کانال مورد نظرش را انتخاب نکرده است و شرط دوم برای وقتی است که کانال مد نظرش را انتخاب کرده است و حالا در استیت های بعدی خواهد بود.

```
if(!channel_command_sent && node()->usart.is_ready_to_send())
{
    Byte selected_channel_byte = static_cast<Byte>(selected_channel);
    node()->usart.write(selected_channel_byte);

    std::cout << "[Microcontroller] requesting sensor on channel " << selected_channel << std::endl;

    channel_command_sent = true;
}
```

داخل ایف اول ما شماره کانال انتخاب شده را تبدیل به یک متغیر تایپ بایت می کنیم تا آن را با پروتکل Usart به ماکس بفرستیم. سپس متغیر آیا کانال انتخاب شد را هم ترو می کنیم.

```
if(channel_command_sent)
{
    if(!node()->usart.has_sensed_zero()) ...

    if(node()->usart.has_sensed_zero() &&
       !node()->usart.has_received_byte() &&
       node()->usart.has_byte_to_read()) ...

    if(node()->usart.has_sensed_zero() &&
       node()->usart.has_received_byte() &&
       !node()->usart.has_sensed_one()) ...

    if(node()->usart.has_sensed_zero() &&
       node()->usart.has_received_byte() &&
       node()->usart.has_sensed_one()) ...
}
```

داخل شرط ایف دوم تقریباً شبیه به مد ۲ است که می خواستیم بایت ها را بخوانیم و نشان دهیم. فقط یک تفاوت بزرگ دارد. در مد ۲ ما صرفاً یک بایت از هارددیسک می خواندیم اما اینجا چون با سنسور کار داریم سنسور ۳ بایت ارسال می کند و نیاز است که مانند بخش اول ما چکسام آن را در نظر بگیریم و از صحت داده دریافتی اطمینان حاصل کنیم.

پس در ایف داخلی اول چک می کنیم صفر دریافت شده یا خیر.

در ایف داخلی دوم چک می کنیم که آیا ۳ بایتی که انتظار داشتیم دریافت شده است یا خیر. اگر دریافت نشده بود باید با پروتکل usart آن را بخوانیم.

```
if(!node()->usart.has_sensed_zero() &&  
!node()->usart.has_received_byte() &&  
node()->usart.has_byte_to_read())  
{  
    received_sensor_packet[received_byte_index] = node()->usart.read();  
}
```

در ایف داخلی سوم منتظر آن می مانیم تا یک را دریافت کنیم تا عملیات تمام شود.

اما ایف آخر مسئولیت بزرگی دارد در قبال اینکه اولاً به تعویض شماره کانال کمک کند و همچنین ۳ بایتی که دریافت کردیم را چک کند که چکسام آن مشکلی نداشته باشد.

```
{  
    node()->usart.reset_receiver_only();  
    received_byte_index++;  
  
    if(received_byte_index == 3)  
    {  
        uint8_t low_byte = static_cast<UByte>(received_sensor_packet[0]);  
        uint8_t high_byte = static_cast<UByte>(received_sensor_packet[1]);  
        uint8_t received_checksum =  
            static_cast<UByte>(received_sensor_packet[2]);  
  
        uint8_t expected_checksum =  
            std::abs(static_cast<int>(low_byte) -  
                static_cast<int>(high_byte));  
  
        if(received_checksum == expected_checksum)  
        {  
            uint16_t sensor_value = (static_cast<uint16_t>(high_byte) << 8) | low_byte;  
  
            std::cout << "[Microcontroller] channel "<< selected_channel<< " value = "<< sensor_value<< " | checksum OK"<< std::endl;  
        }  
        else  
        {  
            std::cout << "[Microcontroller] channel "<< selected_channel<< " checksum FAILED"  
                << " | received="<< static_cast<int>(received_checksum)<< " expected="<< static_cast<int>(expected_checksum)<< std::endl;  
        }  
  
        received_byte_index = 0;  
        selected_channel = 1 - selected_channel;  
        channel_command_sent = false;  
    }  
}
```

عکس ایف داخلی چهارم

در این قسمت ابتدا ۳ بایت را پردازش می کنیم و چک می کنیم که سنسور داده ای را که ارسال کرده در مسیر دچار مشکل نشده باشد و داده صحیح به دست ما رسیده باشد. کد این بخش شبیه به کد مد اول است.

در ادامه تنظیمات میکروکنترلر را برای ارسال دستور بعدی ریست می کنیم. به این صورت که متغیرهای اینکه آیا صفر گرفته یا نه را فالس می کنیم و در پایان ایف هم اولاً شماره کانال را از ۰ به ۱ یا برعکس تغییر می دهیم و اینکه آیا کانال انتخابی ارسال شده است را فالس می کنیم تا مجدد به بیرونی ترین ایف برگردد و همه این مراحل تکرار شود و داده از سنسور کانال جدید تقاضا شود.

I2CMux.h

این بخش ابتدای فایل I2CMux.h است و ساختار سخت افزاری MUX را تعریف می کند.

```
20
21 template <BaudRate BR, BitRate BTR, typename WorkingVoltageTp>
22 requires std::is_base_of_v<AbstractVoltageLevel, WorkingVoltageTp>
23 struct I2CMuxGpio
24 {
25 public:
26     Pins::Vdd<WorkingVoltageTp> vdd {BR, BTR, "I2CMux::vdd"};
27     Pins::Gnd<WorkingVoltageTp> gnd {BR, BTR, "I2CMux::gnd"};
28
29     Pins::Tx<WorkingVoltageTp> TX {BR, BTR, "I2CMux::TX"};
30     Pins::Rx<WorkingVoltageTp> RX {BR, BTR, "I2CMux::RX"};
31
32     Pins::Sda<WorkingVoltageTp> sda0 {BR, BTR, "I2CMux::sda0"};
33     Pins::Scl<WorkingVoltageTp> scl0 {BR, BTR, "I2CMux::scl0"};
34
35     Pins::Sda<WorkingVoltageTp> sda1 {BR, BTR, "I2CMux::sda1"};
36     Pins::Scl<WorkingVoltageTp> scl1 {BR, BTR, "I2CMux::scl1"};
37 };
38
39 class I2CMux : public Board<BaudRates::NotSpecified,
40                             BitRates::same(BaudRates::NotSpecified),
41                             Frequency::F320khz,
42                             I2CMuxVoltage,
43                             I2CMuxGpio>
44 {
45 public:
46     using Parent = Board<BaudRates::NotSpecified,
47                             BitRates::same(BaudRates::NotSpecified),
48                             Frequency::F320khz,
49                             I2CMuxVoltage,
50                             I2CMuxGpio>;
51
52     using Gpio = typename Parent::Gpio;
53 }
```

اول، یک ساختار GPIO برای MUX ساخته شده است. این ساختار مشخص می کند MUX چه پایه هایی دارد و هر پایه چه نقشی در ارتباطات دارد. پایه های vdd و gnd برای تغذیه هستند. پایه های TX و RX برای ارتباط USART با میکروکنترلر استفاده می شوند. یعنی میکروکنترلر از طریق USART به MUX دستور می دهد و MUX هم نتیجه را از همین مسیر برمی گرداند.

بعد از آن، دو جفت پایه ی sda/scl تعریف شده اند. هر جفت نشان دهنده یک کانال جداگانه I2C است. پس sda0/scl0 برای سنسور اول و sda1/scl1 برای سنسور دوم استفاده می شود. دلیل داشتن دو کانال جدا این است که MUX بتواند چند سنسور را جداگانه مدیریت کند و فقط یکی از کانال ها را در هر لحظه فعال کند.

در ادامه، کلاس اصلی I2CMux تعریف می شود که از کلاس پایه ی Board ارث بری می کند. این یعنی MUX در شبیه ساز مثل یک برد سخت افزاری مستقل رفتار می کند؛ درست مثل ESP8266 یا سنسور. برای این برد، نرخ ارتباطی NotSpecified در

نظر گرفته شده، چون جزئیات baudrate/bitrate در این شبیه‌ساز به شکل ساده‌شده مدیریت می‌شود. فرکانس کاری آن هم F320khz است، یعنی MUX خودش clock دارد و می‌تواند به عنوان master سمت I2C عمل کند.

در پایان این بخش، با using Parent نام کلاس پایه کوتاه‌تر شده تا در ادامه فایل راحت‌تر بتوان از آن استفاده کرد. همچنین با using Gpio نوع GPIO مربوط به همین بورد استخراج شده تا در تعریف پروتکل‌ها و توابع بعدی به‌سادگی به پایه‌های MUX دسترسی داشته باشیم.

```

53
54 class I2C : public Protocols::AbstractI2C<I2CMux, Gpio>
55 {
56 public:
57     explicit I2C(I2CMux* board, int channel)
58         : Protocols::AbstractI2C<I2CMux, Gpio> {board},
59           channel {channel}
60     {}
61
62     auto& sda()
63     {
64         if(channel == 0)
65         {
66             return this->m_board->m_gpio.sda0;
67         }
68
69         return this->m_board->m_gpio.sda1;
70     }
71
72     void init(Byte sensor_address) override
73     {
74         pending_sensor_address = sensor_address;
75
76         address_sent = false;
77         waiting_for_ack = false;
78         ack_received = false;
79
80         received_byte_count = 0;
81     }
82
83     void send_address()
84     {
85         write(pending_sensor_address);
86
87         // Release the SDA line so the sensor can send ACK.
88         sda().write(Bit::Z);
89
90         address_sent = true;
91         waiting_for_ack = true;
92     }
93
94     void wait_for_receiving_ack()
95     {
96         if(!sda().hasBitToRead())
97         {
98             return;
99         }
100
101         Bit ack_bit = sda().readBit();
102
103         ack_received = (ack_bit == Bit::One);
104         waiting_for_ack = false;
105
106         if(ack_received)
107         {
108             received_byte_count = 0;
109         }
110     }
111

```

این بخش، کلاس داخلی I2C مربوط به بورد I2CMux را تعریف می‌کند. این کلاس در واقع منطق ارتباط I2C سمت MUX را پیاده‌سازی می‌کند؛ یعنی MUX از طریق این کلاس می‌تواند روی یکی از کانال‌های خودش با سنسور ارتباط برقرار کند.

در constructor، علاوه بر اشاره‌گر به خود بورد MUX، شماره کانال هم دریافت می‌شود. این شماره کانال مشخص می‌کند این instance از I2C مربوط به کدام مسیر فیزیکی است؛ کانال صفر یا کانال یک. بنابراین می‌توان برای هر کانال، یک نمونه جدا از همین کلاس ساخت.

تابع sda وظیفه دارد پایه داده مربوط به کانال فعال را برگرداند. اگر شماره کانال صفر باشد، پایه sda0 استفاده می‌شود و اگر کانال یک باشد، پایه sda1 استفاده می‌شود. این باعث می‌شود بقیه کد بدون تکرار زیاد بتواند با کانال درست کار کند.

تابع init وضعیت اولیه ارتباط با سنسور را آماده می‌کند. در این تابع آدرس سنسوری که قرار است با آن صحبت شود ذخیره می‌شود و flagهای داخلی مربوط به وضعیت ارتباط reset می‌شوند. یعنی مشخص می‌شود هنوز آدرس ارسال نشده، هنوز منتظر ACK نیستیم، ACK دریافت نشده، و هنوز هیچ بیتی از سنسور دریافت نشده است.

تابع send_address اولین مرحله ارتباط I2C با سنسور است. در این مرحله MUX آدرس سنسور را روی خط داده ارسال می‌کند. بعد از ارسال آدرس، خط SDA آزاد می‌شود تا سنسور بتواند پاسخ ACK را روی همان خط قرار دهد. سپس وضعیت داخلی کلاس به‌روزرسانی می‌شود تا مشخص باشد آدرس ارسال شده و MUX حالا منتظر ACK سنسور است.

تابع wait_for_receiving_ack مرحله بعدی ارتباط را مدیریت می‌کند. این تابع بررسی می‌کند آیا روی خط SDA بیتی برای خواندن وجود دارد یا نه. اگر هنوز بیتی آماده نباشد، فعلاً کاری نمی‌کند و در loopهای بعدی دوباره بررسی می‌شود. اگر بیت ACK آماده باشد، آن را می‌خواند و اگر مقدارش One باشد، یعنی سنسور آدرس را پذیرفته است. در این حالت ack_received فعال می‌شود و شمارنده دریافت بایت‌ها هم از صفر شروع می‌شود تا MUX برای خواندن داده‌های سنسور آماده شود.

```

111
112     bool receive_sensor_packet()
113     {
114         if(!sda().hasByteToRead())
115         {
116             return false;
117         }
118
119         Byte received_byte = read();
120         received_sensor_packet[received_byte_count] = received_byte;
121
122         received_byte_count++;
123
124         return received_byte_count == sensor_packet_size;
125     }
126
127     Byte get_received_byte(int index) const
128     {
129         return received_sensor_packet[index];
130     }
131
132     void reset_transaction()
133     {
134         address_sent = false;
135         waiting_for_ack = false;
136         ack_received = false;
137         received_byte_count = 0;
138     }
139
140     void write(Byte byte) override
141     {
142         sda().write(byte);
143     }
144
145     Byte read() override
146     {
147         return sda().read();
148     }
149
150     void run(Gpio&) override
151     {
152     }
153
154     bool has_address_sent() const
155     {
156         return address_sent;
157     }
158
159     bool is_waiting_for_ack() const
160     {
161         return waiting_for_ack;
162     }
163
164     bool has_ack_received() const
165     {
166         return ack_received;
167     }
168

```

این بخش از کلاس I2C مربوط به دریافت داده از سنسور و مدیریت وضعیت ارتباط است. بعد از اینکه MUX آدرس سنسور را فرستاد و ACK را با موفقیت دریافت کرد، سنسور باید سه بایت داده ارسال کند؛ دو بایت اول مقدار تولیدشده توسط سنسور را تشکیل می‌دهند و بایت سوم checksum است. تابع `receive_sensor_packet` بررسی می‌کند که آیا روی خط SDA بایستی برای خواندن وجود دارد یا نه. اگر هنوز بایستی آماده نباشد، مقدار `false` برمی‌گرداند تا در اجرای بعدی `loop` دوباره بررسی شود. اگر بایت آماده باشد، آن را از خط SDA می‌خواند و در آرایه‌ی مربوط به `packet` دریافتی ذخیره می‌کند. سپس شمارنده‌ی بایت‌های دریافت‌شده افزایش پیدا می‌کند. وقتی تعداد بایت‌های دریافتی به اندازه `packet` مورد انتظار برسد، این تابع مقدار `true` برمی‌گرداند؛ یعنی داده‌ی کامل سنسور دریافت شده است.

تابع `get_received_byte` برای دسترسی به بایت‌های ذخیره‌شده استفاده می‌شود. به کمک این تابع، Sketch مربوط به MUX می‌تواند بایت اول، بایت دوم و checksum را جداگانه بردارد و آن‌ها را از طریق USART برای میکروکنترلر ارسال کند. در این مرحله خود MUX مقدار نهایی را محاسبه یا اعتبارسنجی نمی‌کند، بلکه فقط داده‌ی دریافتی از سنسور را به میکروکنترلر منتقل می‌کند.

تابع `reset_transaction` وضعیت ارتباط فعلی I2C را پاک می‌کند تا کانال برای ارتباط بعدی آماده شود. بعد از پایان دریافت داده و ارسال آن به میکروکنترلر، دیگر نباید `flag`های مربوط به ارتباط قبلی باقی بمانند. بنابراین وضعیت‌هایی مثل ارسال شدن آدرس، انتظار برای ACK، دریافت ACK و تعداد بایت‌های دریافتی دوباره `reset` می‌شوند. این کار باعث می‌شود در درخواست بعدی، MUX بتواند دوباره از ابتدا با سنسور موردنظر ارتباط برقرار کند.

توابع `read` و `write` هم واسطه‌ای برای نوشتن و خواندن روی خط SDA همان کانال هستند. وقتی MUX می‌خواهد آدرس سنسور را ارسال کند، از `write` استفاده می‌کند و وقتی می‌خواهد بایت‌های داده را از سنسور بگیرد، از `read` استفاده می‌کند. تابع `run` هم از کلاس پایه پروتکل `override` شده، ولی در این پیاده‌سازی خالی است؛ چون منطق اصلی ارتباط I2C در همین توابع کمکی و در Sketch مربوط به MUX کنترل می‌شود.

در انتهای این بخش چند تابع وضعیت وجود دارد که مشخص می‌کنند ارتباط در چه مرحله‌ای قرار دارد. مثلاً اینکه آیا آدرس ارسال شده، آیا هنوز منتظر ACK هستیم، یا آیا ACK با موفقیت دریافت شده است. این توابع باعث می‌شوند Sketch مربوط به MUX بتواند بدون دسترسی مستقیم به متغیرهای داخلی، وضعیت کانال I2C را بررسی کند و مرحله بعدی ارتباط را انجام دهد.

```

168
169     private:
170         static constexpr int sensor_packet_size = 3;
171
172         int channel {0};
173
174         Byte pending_sensor_address {0};
175         Byte received_sensor_packet[sensor_packet_size] {};
176
177         bool address_sent {false};
178         bool waiting_for_ack {false};
179         bool ack_received {false};
180
181         int received_byte_count {0};
182     };
183
184     mutable I2C i2c0 {this, 0};
185     mutable I2C i2c1 {this, 1};
186

```

این بخش انتهای تعریف کلاس داخلی I2C و همچنین ساخت دو کانال I2C داخل خود MUX است. در قسمت private چند متغیر برای نگهداری وضعیت ارتباط تعریف شده‌اند. مقدار sensor_packet_size برابر ۳ است، چون هر سنسور در این پروژه سه بایت داده ارسال می‌کند: دو بایت برای مقدار اصلی و یک بایت برای checksum. متغیر channel مشخص می‌کند این instance از کلاس I2C مربوط به کدام کانال MUX است؛ کانال صفر یا کانال یک.

متغیر pending_sensor_address آدرس سنسوری را نگه می‌دارد که قرار است روی این کانال با آن ارتباط برقرار شود. قبل از شروع ارتباط، MUX این مقدار را تنظیم می‌کند و بعد همان آدرس را روی باس I2C می‌فرستد. آرایه‌ی received_sensor_packet هم برای ذخیره کردن سه بایت دریافتی از سنسور استفاده می‌شود. وقتی سنسور داده را ارسال می‌کند، بایت‌ها یکی‌یکی در این آرایه قرار می‌گیرند تا packet کامل شود.

سه متغیر address_sent، waiting_for_ack و ack_received وضعیت مراحل مختلف ارتباط را مشخص می‌کنند. address_sent نشان می‌دهد که آیا آدرس سنسور روی I2C ارسال شده یا نه. waiting_for_ack مشخص می‌کند که MUX بعد از ارسال آدرس هنوز منتظر ACK سنسور است یا نه. ack_received هم نشان می‌دهد که ACK با موفقیت از سنسور دریافت شده است یا نه. متغیر received_byte_count تعداد بایت‌هایی را که تاکنون از سنسور دریافت شده‌اند، نگه می‌دارد تا مشخص شود packet سه‌بایتی کامل شده یا هنوز باید منتظر بایت‌های بعدی بمانیم.

در دو خط پایانی، دو نمونه از کلاس I2C ساخته شده‌اند: i2c0 برای کانال صفر و i2c1 برای کانال یک. هر دو با اشاره‌گر به خود MUX ساخته می‌شوند، ولی شماره کانال متفاوت دارند. به این ترتیب MUX می‌تواند دو مسیر I2C جداگانه داشته باشد و بسته به فرمانی که از میکروکنترلر می‌گیرد، یکی از این دو کانال را فعال کند. کلمه‌ی mutable هم باعث می‌شود حتی اگر شیء MUX به صورت const در اختیار Sketch قرار بگیرد، همچنان بتوان وضعیت داخلی کانال‌های I2C را در طول شبیه‌سازی تغییر داد.

```

187 class USART : public Protocols::AbstractUsart<I2CMux, Gpio>
188 {
189 public:
190     explicit USART(I2CMux* board)
191         : Protocols::AbstractUsart<I2CMux, Gpio> {board}
192     {}
193
194     void start()
195     {
196         enabled = true;
197     }
198
199     void write(Byte byte) override
200     {
201         this->m_board->gpio().RX.write(Bit::Zero);
202         this->m_board->gpio().RX.write(byte);
203         this->m_board->gpio().RX.write(Bit::One);
204     }
205
206     Byte read() override
207     {
208         if(this->m_board->gpio().TX.hasByteToRead())
209         {
210             received_frame = this->m_board->gpio().TX.read();
211             received_byte = true;
212             return received_frame;
213         }
214
215         return 0;
216     }
217
218     void set_sensed_zero()
219     {
220         if(this->m_board->gpio().TX.hasBitToRead() &&
221            this->m_board->gpio().TX.readBit() == Bit::Zero)
222         {
223             sensed_zero = true;
224         }
225     }
226
227     void set_sensed_one()
228     {
229         if(this->m_board->gpio().TX.hasBitToRead() &&
230            this->m_board->gpio().TX.readBit() == Bit::One)
231         {
232             sensed_one = true;
233         }
234     }
235
236     bool has_sensed_zero() const
237     {
238         return sensed_zero;
239     }
240
241     bool has_sensed_one() const
242     {
243         return sensed_one;
244     }
245
246     bool has_received_byte() const
247     {
248         return received_byte;
249     }
250

```

این بخش، کلاس داخلی USART مربوط به بورد I2CMux را تعریف می‌کند. این کلاس مسئول ارتباط سریال بین میکروکنترلر و MUX است. یعنی MUX از طریق این USART فرمان را از ESP دریافت می‌کند و بعد از گرفتن داده از سنسور، نتیجه را دوباره از همین مسیر برای ESP می‌فرستد.

در ابتدای کلاس، USART از کلاس پایه‌ی AbstractUsart ارث‌بری کرده است؛ بنابراین باید توابع اصلی یک پروتکل، مثل read و write را پیاده‌سازی کند. constructor این کلاس فقط اشاره‌گر به بورد MUX را می‌گیرد و آن را به کلاس پایه پاس می‌دهد تا USART بتواند به پین‌های TX و RX خود MUX دسترسی داشته باشد.

تابع start برای فعال کردن USART استفاده می‌شود. با صدا زدن این تابع، متغیر داخلی enabled برابر true می‌شود و یعنی ارتباط USART آماده استفاده است. این تابع معمولاً در setup مربوط به Sketch MUX صدا زده می‌شود.

تابع write برای ارسال یک بایت از MUX به سمت میکروکنترلر استفاده می‌شود. ساختار ارسال در اینجا شبیه فریم ساده‌ی USART است: اول یک بیت صفر به عنوان start bit نوشته می‌شود، بعد خود بایت داده ارسال می‌شود، و در نهایت یک بیت یک به عنوان stop bit نوشته می‌شود. در این پیاده‌سازی، MUX برای ارسال داده روی پایه‌ی RX خودش می‌نویسد، چون طبق منطق پروژه، پایه‌های TX/RX در دو طرف ارتباط نسبت به هم معکوس در نظر گرفته شده‌اند.

تابع read برای دریافت یک بایت از سمت میکروکنترلر است. این تابع بررسی می‌کند که آیا روی پایه‌ی TX بایتی برای خواندن وجود دارد یا نه. اگر بایت آماده باشد، آن را می‌خواند، در متغیر received_frame ذخیره می‌کند، وضعیت received_byte را true می‌کند و همان بایت را برمی‌گرداند. اگر هنوز بایتی آماده نباشد، مقدار صفر برگردانده می‌شود.

تابع‌های set_sensed_zero و set_sensed_one برای تشخیص start bit و stop bit در فریم USART استفاده می‌شوند. در USART ساده‌شده‌ی این پروژه، شروع فریم با بیت صفر مشخص می‌شود و پایان فریم با بیت یک. بنابراین set_sensed_zero بررسی می‌کند که آیا بیت صفر از خط ورودی خوانده شده یا نه، و در صورت موفقیت، وضعیت sensed_zero را true می‌کند. به همین شکل، set_sensed_one هم بعد از دریافت داده بررسی می‌کند که آیا بیت یک پایان فریم رسیده یا نه و در صورت مشاهده، sensed_one را true می‌کند.

در انتهای این بخش، چند تابع وضعیت وجود دارد. has_sensed_zero مشخص می‌کند start bit دیده شده یا نه. has_sensed_one مشخص می‌کند stop bit دریافت شده یا نه. has_received_byte هم نشان می‌دهد که بایت اصلی فریم USART خوانده شده است یا هنوز نه. این توابع توسط Sketch مربوط به MUX استفاده می‌شوند تا مرحله‌به‌مرحله بفهمد فریم USART کامل شده یا هنوز باید منتظر ادامه‌ی آن بماند.


```

250
251     void reset_receiver_only()
252     {
253         sensed_zero = false;
254         sensed_one = false;
255         received_byte = false;
256         received_frame = 0;
257     }
258
259     void reset()
260     {
261         reset_receiver_only();
262         enabled = false;
263     }
264
265     void run(Gpio&) override
266     {
267     }
268
269 private:
270     bool enabled {false};
271
272     bool sensed_zero {false};
273     bool sensed_one {false};
274     bool received_byte {false};
275
276     Byte received_frame {0};
277 };
278
279 mutable USART usart {this};
280
281 explicit I2CMux()
282     : Parent {"I2CMux::Processor"}
283 {
284     m_processor->communicationClockChanged.connect(
285         [this](Bit edge) {
286             m_gpio.scl0.nextEdge(edge);
287             m_gpio.scl1.nextEdge(edge);
288         }
289     );
290
291     m_gpio.sda0.write(Bit::Z);
292     m_gpio.sda1.write(Bit::Z);
293
294     m_processor->installProtocol(&usart);
295     m_processor->installProtocol(&i2c0);
296     m_processor->installProtocol(&i2c1);
297
298     std::cout << "one instance of I2CMux created." << std::endl;
299 }
300

```

این بخش انتهایی کلاس USART و سپس constructor اصلی برد I2CMux را نشان می‌دهد. در قسمت اول، دو تابع برای پاک کردن وضعیت USART وجود دارد. تابع `reset_receiver_only` فقط وضعیت دریافت‌کننده را پاک می‌کند؛ یعنی مشخصات مربوط به فریم قبلی مثل دیده شدن `start bit`، دیده شدن `stop bit`، دریافت شدن بایت و مقدار بایت دریافتی را به حالت اولیه برمی‌گرداند. این تابع زمانی کاربرد دارد که یک فریم USART کامل دریافت شده و پردازش شده، اما خود USART همچنان باید فعال بماند تا بتواند فرمان بعدی را هم دریافت کند.

تابع `reset` یک مرحله کامل‌تر است. این تابع اول همان وضعیت `receiver` را پاک می‌کند و سپس `enabled` را هم `false` می‌کند. یعنی ارتباط USART از حالت فعال خارج می‌شود. بنابراین تفاوت این دو تابع این است که `reset_receiver_only` فقط دریافت فریم فعلی را `reset` می‌کند، اما `reset` کل وضعیت USART را غیرفعال می‌کند.

تابع `run` هم از کلاس پایه `override` شده، ولی در این پیاده‌سازی خالی است. دلیلش این است که منطق USART در این پروژه به صورت دستی و مرحله‌به‌مرحله در توابعی مثل `read`، `set_sensed_zero`، `set_sensed_one` و `write` انجام می‌شود و داخل `run` چیزی پیاده‌سازی نشده است.

در بخش `private` چند متغیر وضعیت مربوط به USART تعریف شده‌اند. متغیر `enabled` نشان می‌دهد USART فعال شده یا نه. متغیرهای `sensed_zero` و `sensed_one` به ترتیب مشخص می‌کنند که `start bit` و `stop bit` فریم USART تشخیص داده شده‌اند یا نه. متغیر `received_byte` نشان می‌دهد بایت اصلی فریم دریافت شده است. متغیر `received_frame` هم مقدار همان بایت دریافتی را نگه می‌دارد.

بعد از پایان کلاس USART، یک نمونه از آن داخل خود MUX ساخته می‌شود. با این کار برد I2CMux علاوه بر دو کانال I2C، یک رابط USART هم دارد که از طریق آن با میکروکنترلر ارتباط برقرار می‌کند.

در constructor اصلی I2CMux، ابتدا کلاس پایه `Board` با نام پردازنده‌ی مربوط به MUX مقداردهی می‌شود. سپس `clock` ارتباطی پردازنده به دو پایه‌ی `scl0` و `scl1` وصل می‌شود. این یعنی هر بار `clock` پردازنده تغییر کند، همان تغییر روی هر دو خط SCL مربوط به کانال‌های I2C اعمال می‌شود و سنسورهای متصل به این کانال‌ها می‌توانند با کلاک MUX جلو بروند.

بعد از آن، خطوط `sda0` و `sda1` در حالت آزاد قرار داده می‌شوند. این کار باعث می‌شود باس داده در ابتدا در وضعیت خنثی/آزاد باشد و آماده شروع ارتباط شود. سپس سه پروتکل روی پردازنده MUX نصب می‌شوند: USART، کانال I2C صفر و کانال I2C یک. نصب شدن این پروتکل‌ها یعنی پردازنده شبیه‌ساز آن‌ها را به عنوان اجزای ارتباطی برد می‌شناسد و می‌تواند در جریان اجرای سیستم با آن‌ها کار کند.

در پایان هم یک پیام چاپ می‌شود تا در خروجی مشخص شود که یک نمونه از برد I2CMux با موفقیت ساخته شده است.

I2CMultiplexer.h

این بخش، کلاس I2CMultiplexer را به عنوان Sketch مربوط به سخت افزار I2CMux تعریف می کند. یعنی این کلاس مشخص می کند خود MUX در زمان اجرای شبیه سازی چه رفتاری داشته باشد.

```
class I2CMultiplexer : public AbstractSketch<Comm::I2CMux>
{
public:
    explicit I2CMultiplexer(Comm::I2CMux* node)
        : AbstractSketch<Comm::I2CMux> {node}
    {}

    std::int32_t setup(Comm::I2CMux::Gpio&) override
    {
        std::cout << "[I2C MUX] setup completed." << std::endl;

        node()->usart.start();

        return 0;
    }

    std::int32_t loop(Comm::I2CMux::Gpio&) override
    {
        receive_channel_command();
        handle_sensor_ack();
        handle_sensor_data();

        return 0;
    }
}
```

در constructor، یک اشاره گر به شیء سخت افزاری MUX گرفته می شود و به کلاس پایه AbstractSketch پاس داده می شود. بنابراین از این به بعد داخل Sketch می توانیم با node() به خود MUX و اجزای آن مثل usart و کانال های i2c دسترسی داشته باشیم.

تابع setup فقط یک بار در ابتدای اجرا صدا زده می شود. در اینجا یک پیام چاپ می شود تا مشخص شود MUX راه اندازی شده، سپس با دستور مشخص در کد USART مربوط به MUX فعال می شود.

این کار لازم است چون MUX باید از سمت میکروکنترلر، از طریق USART، فرمان دریافت کند.

تابع loop به صورت مداوم در طول شبیه سازی اجرا می شود. در هر بار اجرای loop، سه مرحله اصلی انجام می شود.

یعنی ابتدا MUX بررسی می کند آیا از میکروکنترلر فرمانی دریافت کرده یا نه. سپس اگر آدرس سنسور را روی I2C فرستاده باشد، منتظر ACK سنسور می ماند. در نهایت اگر ACK دریافت شده باشد، داده های سنسور را می خواند و برای میکروکنترلر ارسال می کند.

```

36 Comm::I2CMux::I2C& active_i2c()
37 {
38     if(selected_channel == 0)
39     {
40         return node()->i2c0;
41     }
42
43     return node()->i2c1;
44 }
45
46 void receive_channel_command()
47 {
48     if(!node()->usart.has_sensed_zero())
49     {
50         node()->usart.set_sensed_zero();
51     }
52
53     if(node()->usart.has_sensed_zero() &&
54        !node()->usart.has_received_byte())
55     {
56         channel_command = node()->usart.read();
57     }
58
59     if(node()->usart.has_sensed_zero() &&
60        node()->usart.has_received_byte() &&
61        !node()->usart.has_sensed_one())
62     {
63         node()->usart.set_sensed_one();
64     }
65
66     if(node()->usart.has_sensed_zero() &&
67        node()->usart.has_received_byte() &&
68        node()->usart.has_sensed_one())
69     {
70         selected_channel =
71             static_cast<int>(static_cast<UByte>(channel_command)) % 2;
72
73         std::cout << "[I2C MUX] selected channel "
74                 << selected_channel
75                 << std::endl;
76
77         active_i2c().init(Sensors::VL530x::address);
78         active_i2c().send_address();
79
80         std::cout << "[I2C MUX] sent sensor address on channel "
81                 << selected_channel
82                 << std::endl;
83
84         waiting_for_sensor_ack = true;
85
86         node()->usart.reset_receiver_only();
87     }
88 }
89

```

۱. تابع `active_i2c` مشخص می‌کند که در حال حاضر کدام کانال I2C فعال است.

اگر مقدار `selected_channel` برابر ۰ باشد، کانال اول یعنی `i2c ۰` برگردانده می‌شود.
اگر مقدار آن ۱ باشد، کانال دوم یعنی `i2c ۱` برگردانده می‌شود.

پس هر جای کد که بنویسیم: `active_i2c()` در واقع داریم با همان کانالی کار می‌کنیم که MUX انتخاب کرده است.

تابع `receive_channel_command`

این تابع مسئول دریافت فرمان از میکروکنترلر از طریق USART است.

میکروکنترلر یک بایت برای MUX می‌فرستد. این بایت مشخص می‌کند که MUX باید کدام کانال I2C را فعال کند.

مرحله اول: تشخیص شروع فریم USART

در USART، شروع فریم با بیت صفر مشخص می‌شود.

اینجا MUX بررسی می‌کند آیا `start bit` را دیده یا نه. اگر هنوز ندیده باشد، تلاش می‌کند آن را تشخیص دهد.

مرحله دوم: خواندن بایت فرمان

بعد از اینکه `start bit` دیده شد، MUX بایت فرمان را از USART می‌خواند و داخل `channel_command` ذخیره می‌کند.

مرحله سوم: تشخیص پایان فریم

بعد از دریافت بایت، MUX منتظر `stop bit` می‌ماند.

در USART، `stop bit` مقدار ۱ دارد. این بخش بررسی می‌کند آیا پایان فریم دیده شده یا نه.

مرحله چهارم: وقتی فریم کامل شد

وقتی `start bit`، بایت داده، و `stop bit` همگی دریافت شدند، یعنی فرمان USART کامل شده است.

مرحله پنجم: انتخاب کانال

اینجا بایت دریافتی تبدیل به شماره کانال می‌شود.

با ۲٪ تضمین می‌شود که شماره کانال فقط ۰ یا ۱ باشد:

بعد هم کانال انتخاب شده چاپ می شود.

مرحله ششم: شروع ارتباط I2C با سنسور

بعد از انتخاب کانال، MUX روی همان کانال I2C ارتباط را شروع می کند.

ابتدا آدرس سنسور را مقداردهی می کند، سپس همان آدرس را روی باس I2C ارسال می کند.

در این لحظه MUX از دید سنسور نقش master را دارد.

مرحله هفتم: رفتن به حالت انتظار ACK

بعد از ارسال آدرس، MUX باید منتظر ACK سنسور بماند.

برای همین این flag را true می کند.

مرحله هشتم: آماده سازی USART برای فرمان بعدی

بعد از اینکه فرمان کانال کامل دریافت و پردازش شد، وضعیت receiver مربوط به USART پاک می شود تا MUX بتواند فرمان بعدی را هم درست دریافت کند.

```

90 void handle_sensor_ack()
91 {
92     if(!waiting_for_sensor_ack)
93     {
94         return;
95     }
96
97     active_i2c().wait_for_receiving_ack();
98
99     if(active_i2c().is_waiting_for_ack())
100     {
101         return;
102     }
103
104     if(active_i2c().has_ack_received())
105     {
106         std::cout << "[I2C MUX] sensor ACK received on channel "
107                     << selected_channel
108                     << std::endl;
109
110         waiting_for_sensor_data = true;
111     }
112     else
113     {
114         std::cout << "[I2C MUX] sensor ACK failed on channel "
115                     << selected_channel
116                     << std::endl;
117     }
118
119     waiting_for_sensor_ack = false;
120 }
121

```

این بخش تابع `handle_sensor_ack()` است و وظیفه‌اش این است که بعد از ارسال آدرس سنسور روی I2C، منتظر پاسخ ACK از سنسور بماند.

این تابع در هر بار اجرای `loop` صدا زده می‌شود، ولی فقط زمانی کاری انجام می‌دهد که MUX واقعاً منتظر ACK سنسور باشد. اگر `waiting_for_sensor_ack` برابر `false` باشد، یعنی هنوز آدرسی برای سنسور ارسال نشده یا مرحله ACK تمام شده است. پس تابع کاری انجام نمی‌دهد و خارج می‌شود.

در ادامه MUX از کانال I2C فعال می‌خواهد که ACK سنسور را بررسی کند.

تابع `active_i2c()` همان کانالی را برمی‌گرداند که قبلاً انتخاب شده بود:

ممکن است در همین `cycle` هنوز بیت ACK آماده نشده باشد.

در این حالت تابع خارج می‌شود و در `loop` بعدی دوباره بررسی می‌کند.

یعنی این بخش باعث می‌شود برنامه عجله نکند و تا آماده شدن ACK صبر کند.

اگر سنسور ACK داده باشد، یعنی آدرس را درست دریافت کرده و آماده ارسال داده است.

در این حالت:

۱. پیام موفقیت چاپ می‌شود.

۲. مقدار `waiting_for_sensor_data` برابر `true` می‌شود.

این یعنی مرحله بعدی شروع می‌شود:

MUX باید سه بایت داده سنسور را بخواند.

اگر ACK دریافت نشود یا مقدار آن درست نباشد، پیام خطا چاپ می‌شود.

یعنی سنسور آدرس را نپذیرفته یا ارتباط I2C درست انجام نشده است.

در پایان، چه ACK موفق باشد چه ناموفق، MUX دیگر در حالت انتظار ACK باقی نمی‌ماند.

اگر ACK موفق بوده باشد، مرحله بعدی با `waiting_for_sensor_data = true` ادامه پیدا می‌کند.

اگر ACK ناموفق بوده باشد، ارتباط برای آن درخواست عملاً تمام می‌شود.


```

121
122 void handle_sensor_data()
123 {
124     if(!waiting_for_sensor_data)
125     {
126         return;
127     }
128
129     if(!active_i2c().receive_sensor_packet())
130     {
131         return;
132     }
133
134     Byte sensor_value_low_byte = active_i2c().get_received_byte(0);
135     Byte sensor_value_high_byte = active_i2c().get_received_byte(1);
136     Byte sensor_checksum = active_i2c().get_received_byte(2);
137
138     node()->usart.write(sensor_value_low_byte);
139     node()->usart.write(sensor_value_high_byte);
140     node()->usart.write(sensor_checksum);
141
142     std::cout << "[I2C MUX] forwarded sensor packet from channel "
143               << selected_channel
144               << std::endl;
145
146     waiting_for_sensor_data = false;
147     active_i2c().reset_transaction();
148 }
149

```

این بخش تابع `handle_sensor_data()` است و وظیفه‌اش این است که بعد از گرفتن ACK از سنسور، داده‌ی سنسور را از I2C بخواند و همان داده را از طریق USART برای میکروکنترلر بفرستد.

اگر `waiting_for_sensor_data` برابر `false` باشد، یعنی هنوز ACK از سنسور نگرفته‌ایم یا اصلاً در مرحله دریافت داده نیستیم. پس تابع کاری انجام نمی‌دهد و خارج می‌شود.

سپس MUX از کانال I2C فعال می‌خواهد packet سنسور را دریافت کند.

سنسور طبق پروژ سه بایت می‌فرستد:

اگر هنوز هر سه بایت کامل دریافت نشده باشد، تابع `receive_sensor_packet()` مقدار `false` برمی‌گرداند و این تابع فعلاً خارج می‌شود.

یعنی در loop بعدی دوباره تلاش می‌کند تا packet کامل شود.

وقتی packet کامل شد، MUX سه بایت دریافتی را جدا می‌کند:

- `sensor_value_low_byte`: بایت کم‌ارزش‌تر عدد سنسور
- `sensor_value_high_byte`: بایت پرارزش‌تر عدد سنسور
- `sensor_checksum`: بایت checksum برای بررسی صحت داده

خود MUX مقدار نهایی را حساب نمی‌کند؛ فقط داده را منتقل می‌کند.

سپس MUX همان سه بایت دریافتی از سنسور را از طریق USART برای میکروکنترلر ارسال می‌کند.

پس نقش MUX در این مرحله بیشتر نقش واسطه است:

پیام چاپ شده نشان می‌دهد که MUX داده سنسور را از کانال انتخاب‌شده گرفته و برای میکروکنترلر ارسال کرده است.

مثلاً اگر کانال ۱ فعال باشد:

بعد از ارسال داده:

- `waiting_for_sensor_data` برابر `false` می‌شود، یعنی مرحله دریافت داده تمام شده است.
- `reset_transaction()` وضعیت کانال I2C را پاک می‌کند تا برای درخواست بعدی آماده شود.

Sensor.h

فایل اسکچ سنسور تقریباً مانند قبل است اما دو قابلیت به سنسور باید اضافه می‌شد. اول اینکه هر سنسور داده متفاوتی از دیگری تولید کند و دوم اینکه یک سنسور پس از اینکه داده را ارسال کرد استیتش را پاک کند و آماده برای ارسال داده بعد در صورت انتخاب مجدد کانالش شود.

```

class Sensor : public AbstractSketch<Sensors::Vl530x>
{
public:
    explicit Sensor(Sensors::Vl530x* sensor_node,
                   int min_value = 0,
                   int max_value = 4000) :
        AbstractSketch<Sensors::Vl530x> {sensor_node},
        min_value {min_value},
        max_value {max_value}
    {
        sensor_node->i2c.set_range(this->min_value, this->max_value);
    }

    std::int32_t setup(Sensors::Vl530x::Gpio&) override ...

    std::int32_t loop(Sensors::Vl530x::Gpio&) override
    {
        if(!node()->i2c.addressMatched)
        {
            node()->i2c.recieve_and_check_address();
        }

        if(node()->i2c.ackSent && !node()->i2c.dataSent)
        {
            node()->i2c.send_3_bytes_of_data();

            if(node()->i2c.dataSent)
            {
                node()->i2c.reset_transaction();
            }
        }

        return 0;
    }
}

```

پس در ابتدا برای اینکه می خواستیم دو سنسور داشته باشیم که یکی یک عدد رندم بین ۰ تا ۲۰ و دیگری عدد رندم بین ۵۰ تا ۱۰۰ تولید کند یک متغیر جدید در این کلاس تعریف می کنیم که مقدار مینیمم و ماکسیمم که سنسور قرار است برگرداند هنگام نمونه برداری از این اسکچ مشخص شود.

همانطور که مشاهده می کنید یک متد `set_range` برای سخت افزاری که سنسور، اسکچ آن است یعنی VL530X ایجاد می کنیم تا این متغیر مینیمم و ماکسیمم به سخت افزار انتقال یابند تا در راستای ساخت اعداد در این بازه به صورت رندم به کار گرفته شوند.

تغییر بعدی بحث ریست کردن سنسور و آماده سازی برای شروع مجدد دریافت آدرس و تولید و ارسال داده است. متد `reset_transaction` وظیفه اش این است که تمام استیت هایی که از سنسور در طول اجرا تغییر کرده است را به حالت اولیه در آورد تا سنسور در حالت آماده باش برای تولید و ارسال داده جدید شود.

```
void reset_transaction()
{
    addressMatched = false;
    ackSent = false;
    dataSent = false;
    txIndex = 0;

    // release I2C data line for next transaction
    m_board->gpio().sda.write(Bit::Z);

    std::cout << "[Sensor] Ready for next transaction" << std::endl;
}
```

final_main.cpp

ما برای گرفتن اجرا از هر بخش نسخه های مختلفی از مین را نوشتیم اما در انتها یک فایل پایانی و جامع نوشتیم که برای تست کردن هریک از سه بخش انجام شده آزادی عمل بیشتری داشته باشیم.

لذا یک فایل برای هر سه اجرا نوشتیم که تنها با تغییر دادن متغیر `mode` و قراردادن آن با مدی که مد نظرمان است جوابی که می خواهیم را مشاهده خواهیم کرد.

```

✓ std::int32_t main()
{
    int mode = 3;
    ✓ // mode = 1 -> I2C only
      // mode = 2 -> USART only
      // mode = 3 -> USART + I2C MUX
    > if(mode == I2C_MODE) ...
    > if(mode == USART_MODE) ...
    > if(mode == I2C_MUX_MODE) ...

    std::cout << "Invalid mode selected." << std::endl;
    return 1;
}

```

ساختار کلی مین به این شکل است که بسته به اینکه کدام مد قرار است اجرا شود تنها یکسری ماژول مرتبط با آن مد ایجاد و راه اندازی می شوند.

در مورد محتوای دو شرط اول تغییری نسبت به توضیحاتی که در بخش های قبل، برای main.cpp هر بخش، داده شد ایجاد نکردیم و تماماً همان ماژول ها ایجاد شدند و صرفاً نامگذاری آن ها تغییر کرد.

اما شرط سوم نمونه برداری های جدید داشت که در ادامه به کد آن می پردازیم.

```

if(mode == I2C_MUX_MODE)
{
    Boards::Esp8266 esp8266;
    Comm::I2CMux i2c_mux;
    Sensors::VL530x sensor_0;
    Sensors::VL530x sensor_1;

    auto power_vdd = std::make_shared<Bus>();
    auto power_gnd = std::make_shared<Bus>();

    auto usart_tx = std::make_shared<Link>();
    auto usart_rx = std::make_shared<Link>();

    auto i2c0_sda = std::make_shared<Link>();
    auto i2c0_scl = std::make_shared<Link>();
    auto i2c1_sda = std::make_shared<Link>();
    auto i2c1_scl = std::make_shared<Link>();

    CPS_SET_OBJECT_NAME(esp8266);
    CPS_SET_OBJECT_NAME(i2c_mux);
    CPS_SET_OBJECT_NAME(sensor_0);
    CPS_SET_OBJECT_NAME(sensor_1);

    CPS_SET_OBJECT_NAME_PTR(power_vdd);
    CPS_SET_OBJECT_NAME_PTR(power_gnd);
    CPS_SET_OBJECT_NAME_PTR(usart_tx);
    CPS_SET_OBJECT_NAME_PTR(usart_rx);
    CPS_SET_OBJECT_NAME_PTR(i2c0_sda);
    CPS_SET_OBJECT_NAME_PTR(i2c0_scl);
    CPS_SET_OBJECT_NAME_PTR(i2c1_sda);
    CPS_SET_OBJECT_NAME_PTR(i2c1_scl);
}

```

در ابتدای کد این بخش نیز مانند مین های پیشین ابتدا سیم ها و باس ها مشخص شده اند و به سیستم شناسانده می شوند و همچنین یک برد، یک ماکس و دو سخت افزار سنسور نمونه گیری می شوند.

```

esp8266.gpio().vdd1.attachLink(power_vdd);
esp8266.gpio().gnd1.attachLink(power_gnd);

i2c_mux.gpio().vdd.attachLink(power_vdd);
i2c_mux.gpio().gnd.attachLink(power_gnd);

sensor_0.gpio().vdd.attachLink(power_vdd);
sensor_0.gpio().gnd.attachLink(power_gnd);

sensor_1.gpio().vdd.attachLink(power_vdd);
sensor_1.gpio().gnd.attachLink(power_gnd);

// USART: ESP <-> I2C MUX
esp8266.gpio().tx.attachLink(usart_tx);
esp8266.gpio().tx.setCanRead(false);
i2c_mux.gpio().TX.attachLink(usart_tx);

i2c_mux.gpio().RX.attachLink(usart_rx);
i2c_mux.gpio().RX.setCanRead(false);
esp8266.gpio().rx.attachLink(usart_rx);

// I2C channel 0 -> sensor 0
i2c_mux.gpio().sda0.attachLink(i2c0_sda);
i2c_mux.gpio().scl0.attachLink(i2c0_scl);
sensor_0.gpio().sda.attachLink(i2c0_sda);
sensor_0.gpio().scl.attachLink(i2c0_scl);

// I2C channel 1 -> sensor 1
i2c_mux.gpio().sda1.attachLink(i2c1_sda);
i2c_mux.gpio().scl1.attachLink(i2c1_scl);
sensor_1.gpio().sda.attachLink(i2c1_sda);
sensor_1.gpio().scl.attachLink(i2c1_scl);

```

در ادامه لینک ها و باس هایی که تعریف کرده بودیم را به پین های هریک از سخت افزارهای موجود متصل می کنیم تا ساختار فیزیکی مدل ما تکمیل شود.

```
MicroController micro_controller(&esp8266, mode);  
I2CMultiplexer i2c_mux_sketch(&i2c_mux);  
  
Sensor sensor_sketch_0(&sensor_0, 0, 20);  
Sensor sensor_sketch_1(&sensor_1, 50, 100);  
  
micro_controller.start();  
i2c_mux_sketch.start();  
sensor_sketch_0.start();  
sensor_sketch_1.start();  
  
return Application::exec();
```

در پایان قرار است به سخت افزارمان روح بدهیم و با استفاده از اسکچ ها آن را به مرحله اجرا برسانیم. پس یک میکروکنترلر که مد آن مشخص شده است ایجاد می کنیم. سپس مالتی پلکسر مربوط و دو سنسور تعریف می کنیم که هر کدام بازه ای در آرگومان می گیرند که داده، تولید و ارسال کنند.

در نهایت هم ۴ ماژول را استارت می زنیم تا شروع به کار کنند.

خروجی زیر از مدل ما به ما نشان می دهد مدل ما توانایی برقراری ارتباط درست با سنسورها و دریافت داده های مجاز از سوی آن ها را دارد.


```
[Microcontroller] requesting sensor on channel 0
[I2C MUX] setup completed.
[Sensor] ready with value range 0 to 20
vl530x setup completed.
[I2C MUX] selected channel 0
[I2C MUX] sent sensor address on channel 0
[Sensor] Address matched!
[Sensor] ACK sent = 1
[Sensor] generated value = 11 range = 0 to 20
[Sensor] Preparing for sending data to Esp.
[Sensor] Sending Byte 0: 11
[I2C MUX] sensor ACK received on channel 0
[Sensor] Sending Byte 1: 0
[Sensor] Sending Byte 2: 11
[Sensor] Transmission finished
[Sensor] Ready for next transaction
[I2C MUX] forwarded sensor packet from channel 0
[Microcontroller] channel 0 value = 11 | checksum OK
[Microcontroller] requesting sensor on channel 1
[I2C MUX] selected channel 1
[I2C MUX] sent sensor address on channel 1
[Sensor] Address matched!
[Sensor] ACK sent = 1
[Sensor] generated value = 90 range = 50 to 100
[Sensor] Preparing for sending data to Esp.
[Sensor] Sending Byte 0: 90
[I2C MUX] sensor ACK received on channel 1
[Sensor] Sending Byte 1: 0
[Sensor] Sending Byte 2: 90
[Sensor] Transmission finished
[Sensor] Ready for next transaction
[I2C MUX] forwarded sensor packet from channel 1
[Microcontroller] channel 1 value = 90 | checksum OK
```

همانطور که مشاهده می کنید میکروکنترلر از سنسور اول عدد ۱۱ و از سنسور دوم عدد ۹۰ را دریافت کرده که در بازه‌ای که
برایش تعریف کردیم قرار دارد.

سوالات

۱. تابع reverse در هدر Byte.h و مکان های استفاده از آن را مطالعه کنید. کارکرد آن را توضیح دهید و
بگویید آیا از همچنین عملیاتی در بوردهای واقعی استفاده می شود ؟ راه حل دنیای واقعی آن چیست ؟(بیان
عمیق مشکل در صورت عدم ارائه راه حل دنیای واقعی نیز قابل پذیرش است)

```
13
14 inline constexpr Byte reverse(Byte b)
15 {
16     Byte res = 0;
17
18     for(std::uint8_t i = 0; i < bitWidth(Byte {}); i++)
19     {
20         auto bit = takeNthBit(b, i);
21         res <<= 1;
22         res += (std::uint8_t)bit;
23     }
24
25     return res;
26 }
```

کارکرد تابع reverse

این تابع ترتیب بیت‌های یک بایت را برعکس می‌کند.

یعنی بیت کم‌ارزش‌تر، یا همان LSB، را تبدیل می‌کند به بیت پرارزش‌تر، یا MSB.

مثلاً اگر بایت ورودی این باشد: 00010110

خروجی می‌شود: 01101000

چون ترتیب بیت‌ها کاملاً برعکس شده است.

چرا چنین تابعی لازم شده؟

در شبیه‌ساز هنگام نوشتن یک بایت روی لینک، بیت‌ها به یک ترتیب خاص در buffer ذخیره می‌شوند. اما هنگام خواندن، اگر همان ترتیب مستقیم خوانده شود، ممکن است مقدار نهایی برعکس چیزی شود که فرستنده قصد ارسال آن را داشته.

برای همین در تابع read() کلاس Transmitter این را داریم:

```
250 inline Byte
251 read()
252 {
253     std::scoped_lock lock(m_readMutex);
254     auto first = m_receivingBuffer.takeRest();
255     first      = reverse(first);
256     return first;
257 }
258
```

یعنی:

۱. بیت‌های دریافت‌شده از buffer برداشته می‌شوند.
۲. چون ترتیب ذخیره‌شدن بیت‌ها با ترتیب عددی بایت فرق دارد، reverse روی آن اعمال می‌شود.
۳. در نهایت بایت درست به لایه بالاتر تحویل داده می‌شود.

پس reverse در این پروژه برای اصلاح ترتیب بیت‌های دریافتی استفاده شده است.

آیا در بوردهای واقعی هم از چنین عملیاتی استفاده می‌شود؟

در دنیای واقعی، خود سخت‌افزارهای ارتباطی مثل UART، I2C، SPI و کنترلرهای داخلی میکروکنترلر معمولاً ترتیب ارسال و دریافت بیت‌ها را مدیریت می‌کنند.

مثلاً در UART معمولاً بیت‌ها به صورت LSB first ارسال می‌شوند. اما سخت‌افزار UART خودش می‌داند که این بیت‌ها را چگونه جمع کند تا در نهایت همان بایت صحیح در رجیستر داده قرار بگیرد.

بنابراین در حالت معمول، برنامه‌نویس لازم نیست خودش بیت‌های بایت را reverse کند.

پس آیا اصل این کار واقعی است؟

خود عملیات برعکس کردن بیت‌ها در دنیای واقعی وجود دارد، ولی معمولاً در این شرایط:

- وقتی داده از یک پروتکل با ترتیب بیت متفاوت می‌آید.
- وقتی با hardware peripheral خاصی کار می‌کنیم که bit order متفاوت دارد.
- وقتی داریم پروتکل را به صورت نرم‌افزاری یا bit-banging پیاده‌سازی می‌کنیم.
- وقتی داده از network / sensor / peripheral با endian یا bit order متفاوت می‌آید.
- در کارهای low-level مثل protocol adapter و CRC ، encoder/decoder.driver

اما در یک UART یا I2C واقعی استاندارد، این اصلاح معمولاً توسط سخت‌افزار یا peripheral انجام می‌شود، نه توسط کد application.

راه‌حل دنیای واقعی چیست؟

در دنیای واقعی معمولاً یکی از این راه‌ها استفاده می‌شود:

۱. تنظیم سخت‌افزار peripheral

مثلاً بعضی peripheral اجازه می‌دهند مشخص کنیم داده MSB first یا LSB first منتقل شود.

۲. استفاده از رجیسترهای سخت‌افزاری

در UART/I2C واقعی، سخت‌افزار بیت‌ها را جمع می‌کند و در نهایت یک بایت درست داخل data register قرار می‌دهد.

۳. استفاده از shift register

در سطح سخت‌افزار، بیت‌ها وارد shift register می‌شوند و در پایان یک بایت کامل ساخته می‌شود.

۴. اصلاح نرم‌افزاری در driver

اگر سخت‌افزار چنین قابلیت‌ای نداشته باشد، driver ممکن است مثل همین پروژه بیت‌ها را reverse کند.

جمع‌بندی سؤال ۱

تابع reverse ترتیب بیت‌های یک Byte را برعکس می‌کند. در این پروژه در تابع read() استفاده شده تا بایستی که از buffer بیت‌ها ساخته شده، با ترتیب صحیح به برنامه تحویل داده شود. در بوردهای واقعی، چنین مشکلی معمولاً توسط سخت‌افزارهای ارتباطی مثل UART/I2C حل می‌شود، اما در پیاده‌سازی نرم‌افزاری پروتکل یا شبیه‌سازی، ممکن است لازم باشد این عملیات دستی انجام شود.

۲.تابع takeNthBit در Bit.h را توضیح دهید. تفاوت این دو در چیست ؟

```
33
34 template <typename T>
35 requires std::is_integral_v<T>
36 inline constexpr Bit
37 takeNthBit(T value, std::uint8_t n)
38 {
39     if(bitWidth<T>() <= n)
40     {
41         std::cerr << "n is larger than bit width of provided value " << (int)n
42         << " >= " << (int)bitWidth<T>() << std::endl;
43         return Bit::X;
44     }
45
46     T mask = 1;
47     mask <<= n;
48     T result = value & mask;
49
50     if(result)
51         return Bit::One;
52     else
53         return Bit::Zero;
54 }
55
56
```

کارکرد نسخه اول

این تابع بیت شماره n از مقدار value را برمی گرداند.

مثلاً اگر داشته باشیم: $value = 0b\ 10100000$

بیت‌ها از سمت راست شماره گذاری می شوند:

bit 0 = 0

bit 1 = 1

bit 2 = 0

bit 3 = 1

پس خروجی takeNthBit(value) ۱ می شود Bit::One و خروجی takeNthBit(value) ۲ می شود Bit::Zero

این تابع چگونه کار می کند؟

اول یک mask می سازد:

T mask = 1;

mask <<= n;

`.....\..... = mask`

یعنی اگر $n = 3$ باشد:

`T result = value & mask`

بعد:

اگر نتیجه صفر نبود، یعنی بیت n ام برابر ۱ است.

اگر نتیجه صفر بود، یعنی بیت n ام برابر ۰ است.

نکته مهم نسخه اول

در این نسخه مقدار n در زمان اجرا داده می‌شود.

یعنی می‌توانیم بنویسیم:

```
std::uint8_t i = 3;
```

```
takeNthBit(byte, i);
```

پس این تابع برای loopها مناسب است.

مثلاً در reverse همین اتفاق افتاده:

```
for(std::uint8_t i = 0; i < bitWidth<Byte>(); i++)
```

```
{ auto bit = takeNthBit(b, i);}
```

اینجا i در زمان اجرا تغییر می‌کند، پس باید از نسخه اول استفاده شود.

```

56
57 template <auto byte, std::uint8_t n>
58 requires std::is_integral_v<decltype(byte)>
59 inline constexpr Bit
60 takeNthBit()
61 {
62     using T = decltype(byte);
63
64     static_assert(bitWidth<T>() >= n,
65                 "n is larger than bit width of provided value");
66
67     T mask = 1;
68     mask <= n;
69     T result = byte & mask;
70
71     if(result)
72         return Bit::One;
73     else
74         return Bit::Zero;
75 }
76

```

کارکرد نسخه دوم

این هم همان کار را انجام می‌دهد: بیت شماره n از byte را برمی‌گرداند.

اما فرقی این است که مقدار byte و n به عنوان template parameter داده می‌شوند.

یعنی استفاده‌اش این شکلی است:

```
takeNthBit<0b00001010, 1>()
```

نه این شکلی:

```
takeNthBit(value, 1)
```

تفاوت اصلی دو نسخه

نسخه اول: runtime

در نسخه اول، مقدار ورودی و شماره بیت در زمان اجرا مشخص می‌شوند: takeNthBit(value, n)

ویژگی‌ها:

- مناسب برای مقادیری که هنگام اجرای برنامه معلوم می‌شوند.
- مناسب برای loop.

- اگر n اشتباه باشد، پیام خطا چاپ می‌کند و `Bit::X` برمی‌گرداند.
- انعطاف‌پذیرتر است.

نسخه دوم: `compile-time`

در نسخه دوم، مقدار و شماره بیت در زمان کامپایل مشخص‌اند:

`takeNthBit<byte, n>()`

ویژگی‌ها:

- مناسب برای مقادیر ثابت.
- می‌تواند در زمان کامپایل محاسبه شود.
- اگر n معتبر نباشد، با `static_assert` خطای کامپایل می‌دهد.
- از نظر `type-safety` و `performance` بهتر است، ولی انعطاف کمتری دارد.

تفاوت در خطایابی

`return Bit::X`

در نسخه اول اگر n از عرض بیت بیشتر باشد:

یعنی برنامه همچنان اجرا می‌شود، ولی بیت نامعتبر برمی‌گردد.

`static_assert (...)`

اما در نسخه دوم:

یعنی اصلاً اجازه کامپایل نمی‌دهد.

جمع‌بندی سؤال ۲

هر دو تابع `takeNthBit` بیت n ام یک عدد صحیح را استخراج می‌کنند. نسخه اول مقدار و شماره بیت را در زمان اجرا می‌گیرد و برای `loop`ها و داده‌های متغیر مناسب است. نسخه دوم مقدار و شماره بیت را در زمان کامپایل می‌گیرد و برای مقادیر ثابت مناسب است. تفاوت اصلی آن‌ها در زمان ارزیابی، نوع خطایابی، و کاربردها است.

۳. تابع `attachPinToCommunicationClock` از کلاس `Board` توضیح دهید (از جهت کد و کاربرد)


```

106     attachPinToCommunicationClock()
107     {
108         static_assert(frequency() != Frequency::Drived,
109             "failed to attach pin to communication clock,"
110             "you can not call function "
111             "'attachPinToCommunicationClock' on a slave board.");
112
113         static_assert(GpioHelper::field_count_v<Gpio> > pinIndex,
114             "failed to attach pin to communication clock, "
115             "invalid pin index.");
116
117         auto& pin = boost::pfr::get<pinIndex>(m_gpio);
118
119         static_assert(std::is_same_v<Pins::Digital<WorkingVoltageTp>,
120             std::decay_t<decltype(pin)>>,
121             "failed to attach pin to communication clock, selected "
122             "pin type must be Digital.");
123
124         m_processor->communicationClockChanged.connect(
125             [this, &pin](Bit edge) { pin.write(edge); });
126     }
127

```

کار کلی تابع

این تابع یک پین دیجیتال از GPIO برد را به clock داخلی پردازنده وصل می‌کند.

یعنی هر وقت clock پردازنده تغییر کند، مقدار همان clock روی آن pin نوشته می‌شود.

به زبان ساده:

این تابع باعث می‌شود یک pin دیجیتال مثل خروجی clock رفتار کند.

کاربردش در پروژه

در شبیه‌ساز، پردازنده یک signal دارد به نام: communicationClockChanged

هر بار clock تغییر می‌کند، این signal یک edge تولید می‌کند.

تابع attachPinToCommunicationClock یک pin را به این signal وصل می‌کند:

```
m_processor->communicationClockChanged.connect([this, &pin](Bit edge) { pin.write(edge); });
```

یعنی تغییر کلاک پردازنده باعث تولید edge میشود و روی پین انتخاب شده نوشتن انجام می‌شود.

چرا چنین چیزی لازم است؟

بعضی پروتکل‌ها مثل I2C/C2I پایه clock دارند.

در I2C، پایه SCL برای هماهنگ کردن فرستنده و گیرنده استفاده می‌شود.

در این پروژه هم طبق توضیح تمرین، پایه SCL به صورت خودکار به clock میکروکنترلر وصل می‌شود و ما مستقیماً موج clock را کنترل نمی‌کنیم.

این تابع دقیقاً برای همین ایده است:

یک pin را به clock ارتباطی بورد وصل می‌کند.

بررسی خط به خط تابع

۱. جلوگیری از استفاده روی slave board

```
static_assert(frequency() != Frequency::Drived, ...);
```

اگر فرکانس بورد Drived باشد، یعنی این بورد clock مستقل ندارد و از بورد دیگری clock می‌گیرد.

مثلاً سنسور VL530X در کد: Frequency::Drived یعنی خودش clock تولید نمی‌کند.

پس اجازه ندارد بگوید «یک pin من را به clock داخلی وصل کن»، چون clock داخلی مستقلی ندارد.

بنابراین این تابع فقط برای master board معنا دارد.

۲. بررسی معتبر بودن شماره pin

```
static_assert(GpioHelper::field_count_v<Gpio> > pinIndex, ...);
```

این بررسی می‌کند که pinIndex از تعداد پین‌های GPIO بیشتر نباشد.

مثلاً اگر GPIO شانزده pin دارد، نباید pin شماره ۲۰ را انتخاب کنیم.

۳. گرفتن pin با boost::pfr

```
auto& pin = boost::pfr::get<pinIndex>(m_gpio);
```

اینجا از boost::pfr استفاده شده تا عضو شماره pinIndex از struct مربوط به GPIO گرفته شود.

مثلاً اگر pinIndex برابر ۱۰ باشد، عضو دهم GPIO گرفته می‌شود.

۴. بررسی اینکه pin حتماً دیجیتال باشد

```
static_assert(std::is_same_v<Pins::Digital<WorkingVoltageTp>, std::decay_t<decltype(pin)>>, ...);
```

این بخش می گوید فقط pin از نوع Digital قابل اتصال به clock است.

یعنی نمی توانی مثلاً Scl، Sda، Rx، Tx یا این تابع وصل کنی؛ این تابع مخصوص pin های دیجیتال عمومی است.

۵. اتصال pin به clock

```
m_processor->communicationClockChanged.connect([this, &pin](Bit edge) { pin.write(edge); });
```

این بخش اصل کار را انجام می دهد.

هر بار که clock تغییر کند، مقدار edge روی pin نوشته می شود:

pin.write(edge)

از جهت کاربردی

فرض کنیم می خواهیم pin دیجیتال d0 برد را به clock پردازنده وصل کنیم.

با این تابع، هر بار clock پردازنده ۰ و ۱ می شود، pin d0 هم ۰ و ۱ می شود.

پس می توان از آن به عنوان یک clock output برای راه اندازی یک قطعه دیگر استفاده کرد.

چرا روی slave board ممنوع است؟

چون slave clock تولید نمی کند؛ فقط clock را از master دریافت می کند.

مثلاً در I2C:

- میکروکنترلر master است و SCL را تولید می کند.
- سنسور slave است و SCL را دریافت می کند.

اگر سنسور هم بخواهد clock تولید کند، تداخل ایجاد می شود.

جمع بندی سؤال ۳

تابع attachPinToCommunicationClock یک pin دیجیتال از GPIO برد را به clock ارتباطی پردازنده وصل می کند. با این کار هر edge از clock پردازنده روی آن pin نوشته می شود. این تابع فقط روی بوردهایی مجاز است که clock مستقل دارند،

نه بوردهایی با فرکانس Drived. همچنین فقط برای pinهای Digital قابل استفاده است. کاربرد اصلی آن در شبیه‌سازی انتشار clock از برد master به یک pin خروجی است.